Marketing Data Analysis

a. Preprocess and clean if necessary.
b. Build a model predicting “churn”.
c. Remember to comment your code and give rationales for models, algorithms, and approaches.

Import Packages

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# %matplotlib inline
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)
pd.set_option('display.width', 1000)

import warnings
warnings.filterwarnings("ignore")

import os
# path=os.environ['USERPROFILE']+r'\OneDrive\BDA2'
path=os.environ['USERPROFILE']+r'\Documents\BDA2'
```

Load data

In [2]:
```
import pyodbc
import urllib
import sqlalchemy

'''connect to datahub'''

params_datahub = urllib.parse.quote_plus("DRIVER={SQL Server Native Client 11.0};"
                                 "SERVER=localhost\SQLEXPRESS;"
                                 "DATABASE=datahub;"
                                 "UID=sa;"
                                 "PWD=user1")

engine_datahub = sqlalchemy.create_engine("mssql+pyodbc:///?odbc_connect={}".format(params_datahub
```

In [4]:
```
df=pd.read_sql_table(r"marketing_churn",engine_datahub)
# df=pd.read_excel("C:\Business_Data_Analysis\data\marketing_churn.xlsx")
df.head()
#dataframe: df
# a*x1, b*x2...= y (Exited)  model
# a*x1+b*x2... = probability of Exited
```

Out[4]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	H
0	7	15592531	Bartlett	822	France	Male	50	7	0.0	2	

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	H
1	21	15577657	McDonald	732	France	Male	41	8	0.0	2	
2	77	15614049	Hu	664	France	Male	55	8	0.0	2	
3	94	15640635	Capon	769	France	Male	29	8	0.0	2	
4	142	15724944	Tien	663	France	Male	34	7	0.0	2	

Exploratory Data Analysis(EDA)

Check missing values and shape

Normally we need to clean the samples, i,e, impute missing values but in this case the data is pretty clean with no missing values. We also check the shape to make sure it matches the meta data info in the document.

In [5]:
```
df.info(), df.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
(None, (10000, 14))
```

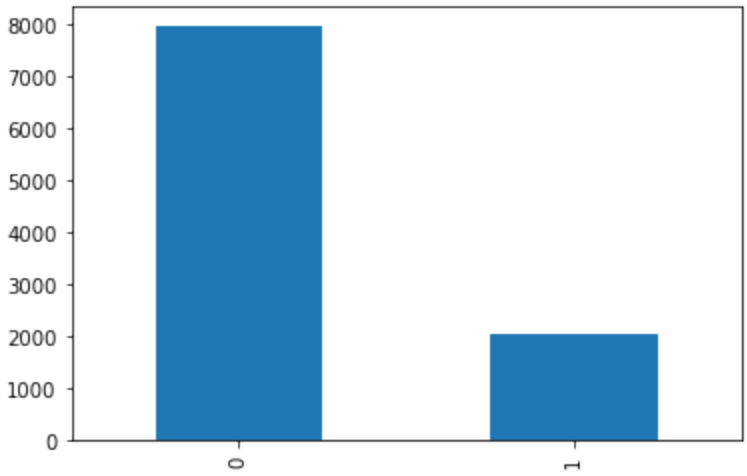
Out[5]:

Check Churn ratio

The samples are balanced so we can use "Accuracy" metric to measure the performance of the model

In [7]:
```
target='Exited'
df[target].value_counts().plot(kind='bar')
```

Out[7]:
```
<AxesSubplot:>
```



Describe the data

Categorical features/Dimensions

In [8]:
```
droplist='Surname'
cat_cols=df.select_dtypes(object).drop(droplist,axis=1).columns.tolist()
cat_cols
```

Out[8]:
```
['Geography', 'Gender']
```

In [9]:
```
df.select_dtypes(object).drop(droplist,axis=1).columns.tolist()
```

Out[9]:
```
['Geography', 'Gender']
```

In [12]:
```
help(df.select_dtypes)
```

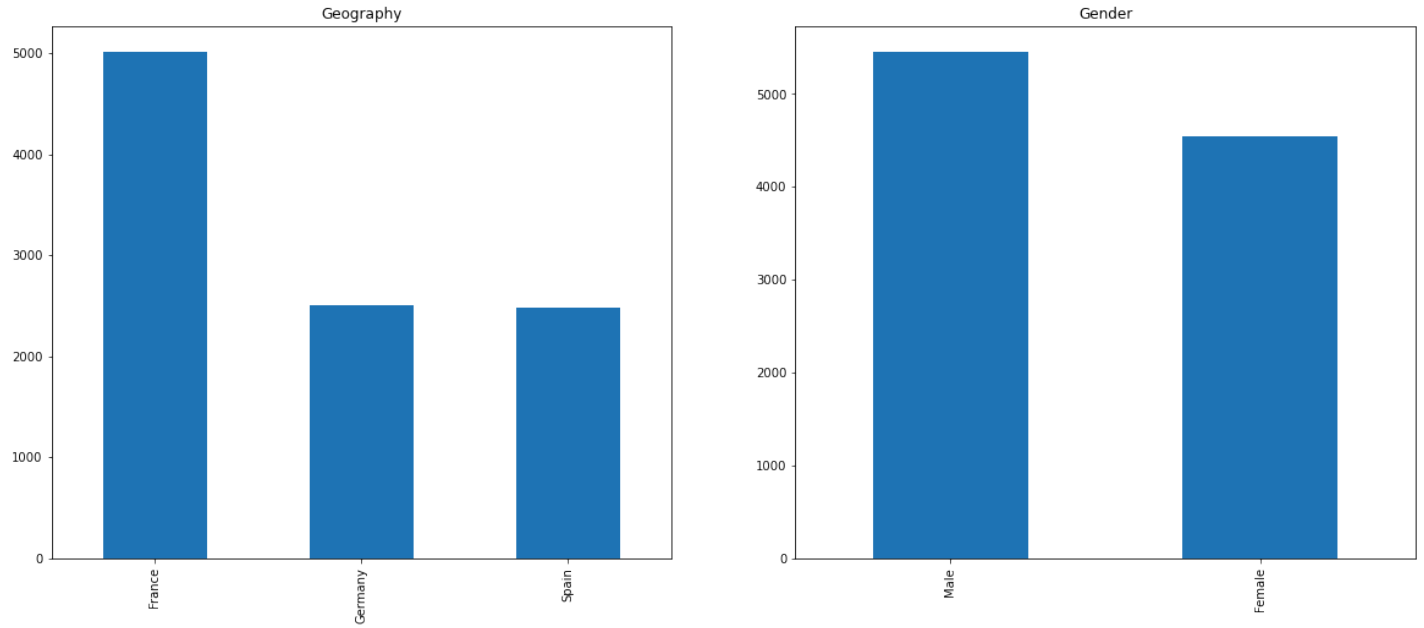
```
Help on method select_dtypes in module pandas.core.frame:

select_dtypes(include=None, exclude=None) -> 'DataFrame' method of pandas.core.frame.DataFrame ins
tance
    Return a subset of the DataFrame's columns based on the column dtypes.

    Parameters
    ----------
    include, exclude : scalar or list-like
        A selection of dtypes or strings to be included/excluded. At least
        one of these parameters must be supplied.

    Returns
    -------
    DataFrame
        The subset of the frame including the dtypes in ``include`` and
        excluding the dtypes in ``exclude``.

    Raises
    ------
    ValueError
        * If both of ``include`` and ``exclude`` are empty
        * If ``include`` and ``exclude`` have overlapping elements
        * If any kind of string dtype is passed in.

    See Also
```

```
--------
DataFrame.dtypes: Return Series with the data type of each column.

Notes
-----
* To select all *numeric* types, use ``np.number`` or ``'number'``
* To select strings you must use the ``object`` dtype, but note that
  this will return *all* object dtype columns
* See the `numpy dtype hierarchy
  <https://numpy.org/doc/stable/reference/arrays.scalars.html>`__
* To select datetimes, use ``np.datetime64``, ``'datetime'`` or
  ``'datetime64'``
* To select timedeltas, use ``np.timedelta64``, ``'timedelta'`` or
  ``'timedelta64'``
* To select Pandas categorical dtypes, use ``'category'``
* To select Pandas datetimetz dtypes, use ``'datetimetz'`` (new in
  0.20.0) or ``'datetime64[ns, tz]'``

Examples
--------
>>> df = pd.DataFrame({'a': [1, 2] * 3,
...                    'b': [True, False] * 3,
...                    'c': [1.0, 2.0] * 3})
>>> df
        a      b  c
0       1   True  1.0
1       2  False  2.0
2       1   True  1.0
3       2  False  2.0
4       1   True  1.0
5       2  False  2.0

>>> df.select_dtypes(include='bool')
   b
0  True
1  False
2  True
3  False
4  True
5  False

>>> df.select_dtypes(include=['float64'])
   c
0  1.0
1  2.0
2  1.0
3  2.0
4  1.0
5  2.0

>>> df.select_dtypes(exclude=['int64'])
       b    c
0   True  1.0
1  False  2.0
2   True  1.0
3  False  2.0
4   True  1.0
5  False  2.0
```

In [10]:

```
import matplotlib.pyplot as plt
categorical_features = cat_cols
```

```
fig, ax = plt.subplots(1, len(categorical_features))

for i, categorical_feature in enumerate(df[categorical_features]):
    df[categorical_feature].value_counts().plot(kind="bar", ax=ax[i],figsize=(20,8),rot=90,fontsiz
```



Numeric data

Histogram

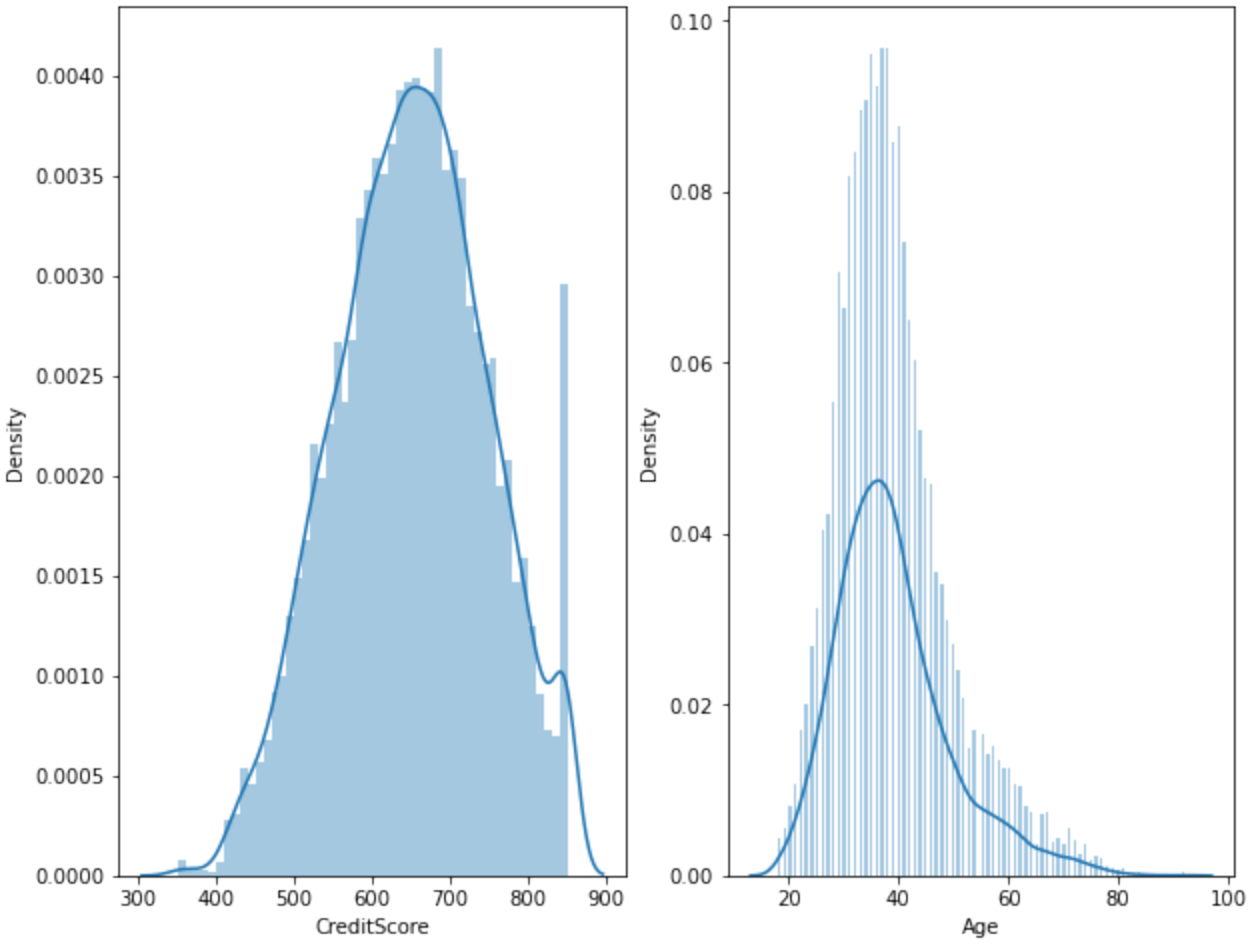
Histogram groups numeric data into bins, displaying the bins as segmented columns and summarize the distribution of a univariate data set.

In [11]:
```
droplist=['RowNumber','CustomerId']
num_cols=df.select_dtypes('number').drop(droplist,axis=1).columns.tolist()
num_cols
```

Out[11]:
```
['CreditScore',
 'Age',
 'Tenure',
 'Balance',
 'NumOfProducts',
 'HasCrCard',
 'IsActiveMember',
 'EstimatedSalary',
 'Exited']
```

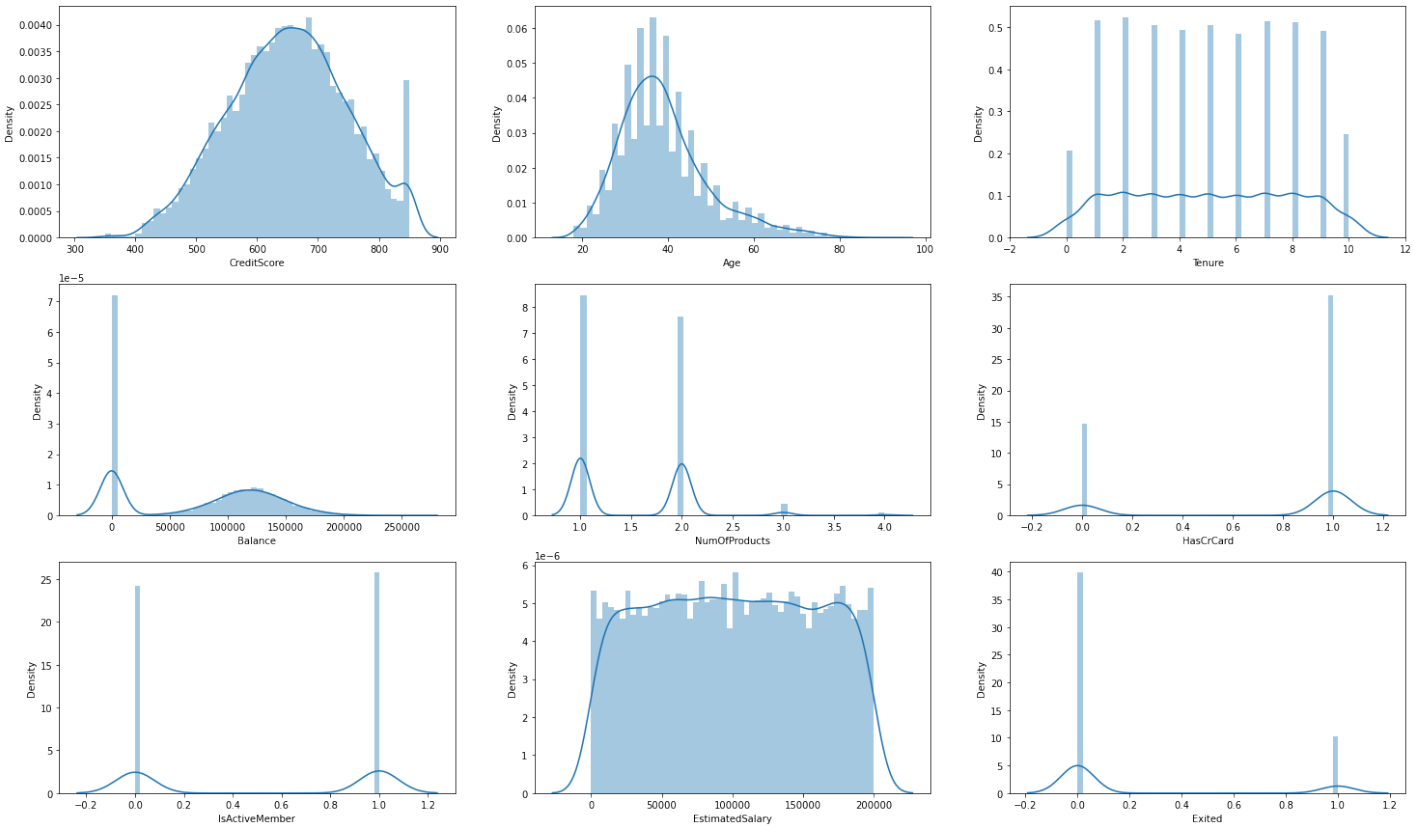
In [12]:
```
fig, ax = plt.subplots(1,2, figsize=(10,8))

sns.distplot(df['CreditScore'], ax=ax[0],bins=50)
sns.distplot(df['Age'],ax=ax[1], bins=150)
```

Out[12]:
```
<AxesSubplot:xlabel='Age', ylabel='Density'>
```



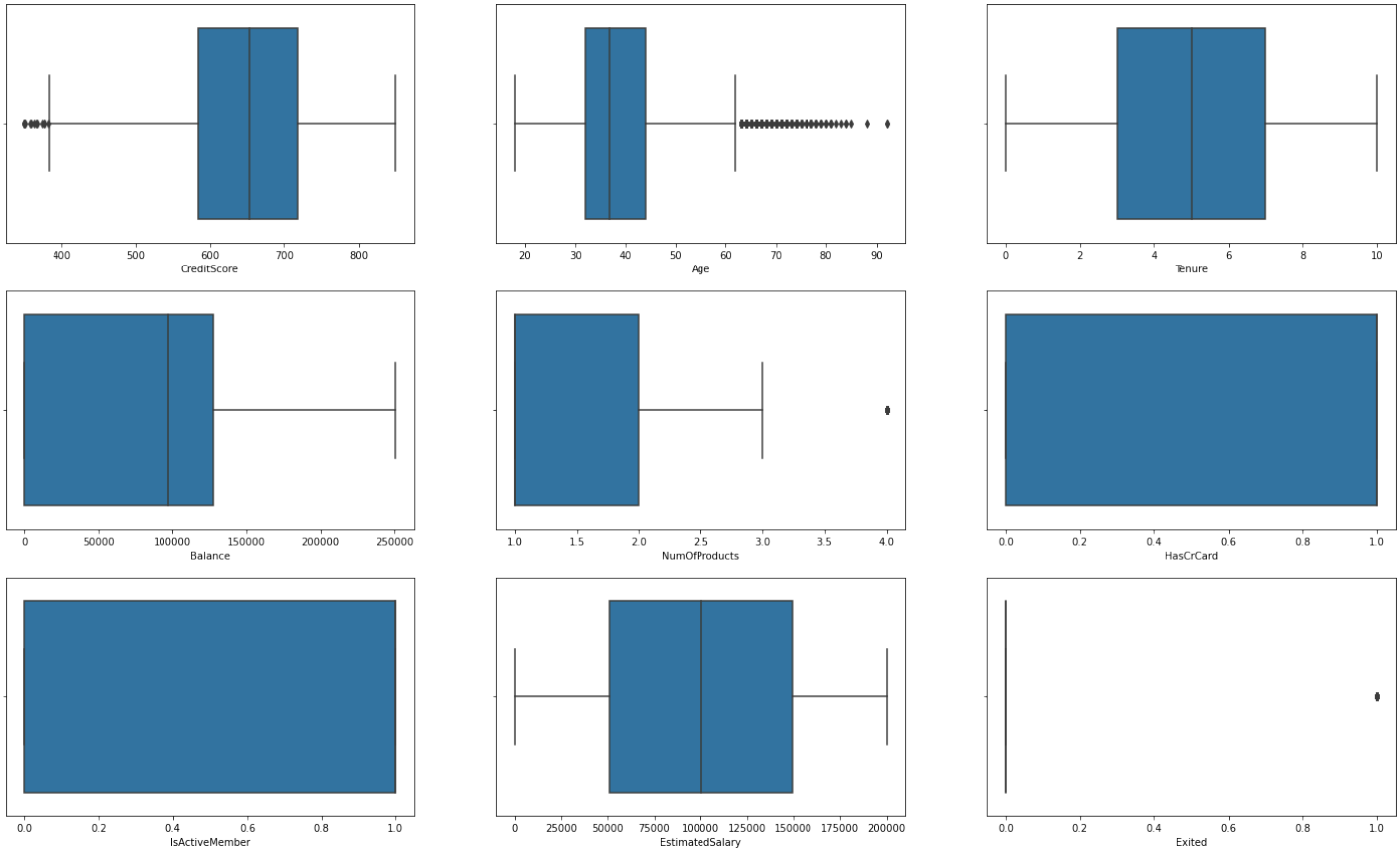
In [16]:

```
# import warnings
# warnings.simplefilter(action='ignore', category=FutureWarning)

fig, ax = plt.subplots(3,3, figsize=(25,15))

for i in range(ax.shape[0]):
    for j in range(ax.shape[1]):
            sns.distplot(df[df[num_cols].columns[i*3+j]], ax=ax[i][j],bins=50)


# for i in range(9):
#     print(i)
#     sns.distplot(df[df[num_cols].columns[i]], ax=ax[i],bins=50)
```



In [17]:

```
import warnings
warnings.simplefilter(action='ignore', category=UserWarning)

fig, ax = plt.subplots(3, 3, figsize=(25, 15))

#plot the features except LOCATION_ID and Risk
for i in range(ax.shape[0]):
    for j in range(ax.shape[1]):
#         sns.boxplot(df[df[num_cols].columns[i*2+j]], ax=ax[i][j],orient='v',showfliers=False)
        sns.boxplot(df[df[num_cols].columns[i*3+j]], ax=ax[i][j],orient='v')
```



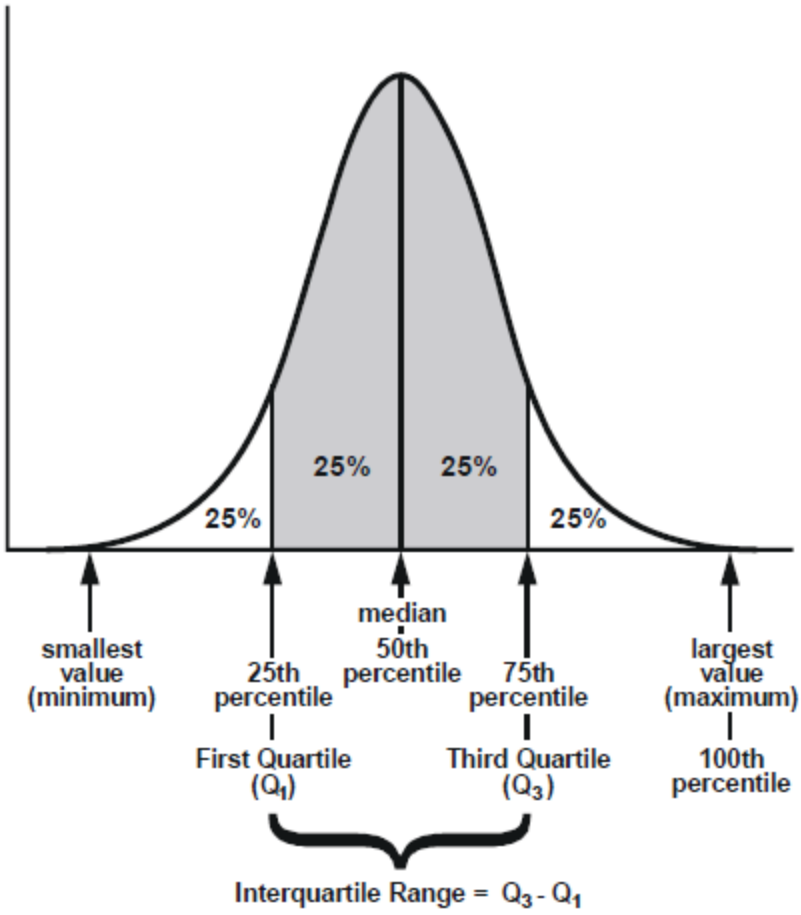
In [18]:
```
df.describe()
# from the "max" row we can see feature PARA_A, PARA_B,Money_Value,History have some potential out
```

Out[18]:

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasC
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.(
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.7
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.4
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.(
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.(
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.(
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.(
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.(

Clip, i.e. assigns values outside boundary to boundary values, the data to deal with outliers.

Outliers may distort how we see the data. They contain information too so it's a tradeoff; we lose some info but gain a better big picture of the data.



In [19]:
```
df[num_cols].clip(lower=df[num_cols].quantile(0.05), upper=df[num_cols].quantile(0.95),axis=1)
```

Out[19]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	812	50	7	0.00	2	1	1	10062.80	0
1	732	41	8	0.00	2	1	1	170886.17	0
2	664	55	8	0.00	2	1	1	139161.64	0
3	769	29	8	0.00	2	1	1	172290.61	0
4	663	34	7	0.00	2	1	1	180427.24	0
...	...	...	...	...	...	...	...	...	...
9995	750	38	5	151532.40	1	1	1	46555.15	0
9996	670	33	8	126679.69	1	1	1	39451.09	0
9997	739	58	2	101579.28	1	1	1	72168.53	0
9998	623	48	5	118469.38	1	1	1	158590.25	0
9999	516	35	9	57369.61	1	1	1	101699.77	0

10000 rows × 9 columns

In [20]:
```
df[num_cols]
```

Out[20]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	822	50	7	0.00	2	1	1	10062.80	0
1	732	41	8	0.00	2	1	1	170886.17	0
2	664	55	8	0.00	2	1	1	139161.64	0
3	769	29	8	0.00	2	1	1	172290.61	0
4	663	34	7	0.00	2	1	1	180427.24	0
...	...	...	...	...	...	...	...	...	...
9995	750	38	5	151532.40	1	1	1	46555.15	0
9996	670	33	8	126679.69	1	1	1	39451.09	0
9997	739	58	2	101579.28	1	1	1	72168.53	0
9998	623	48	5	118469.38	1	1	1	158590.25	0
9999	516	35	10	57369.61	1	1	1	101699.77	0

10000 rows × 9 columns

In [21]:

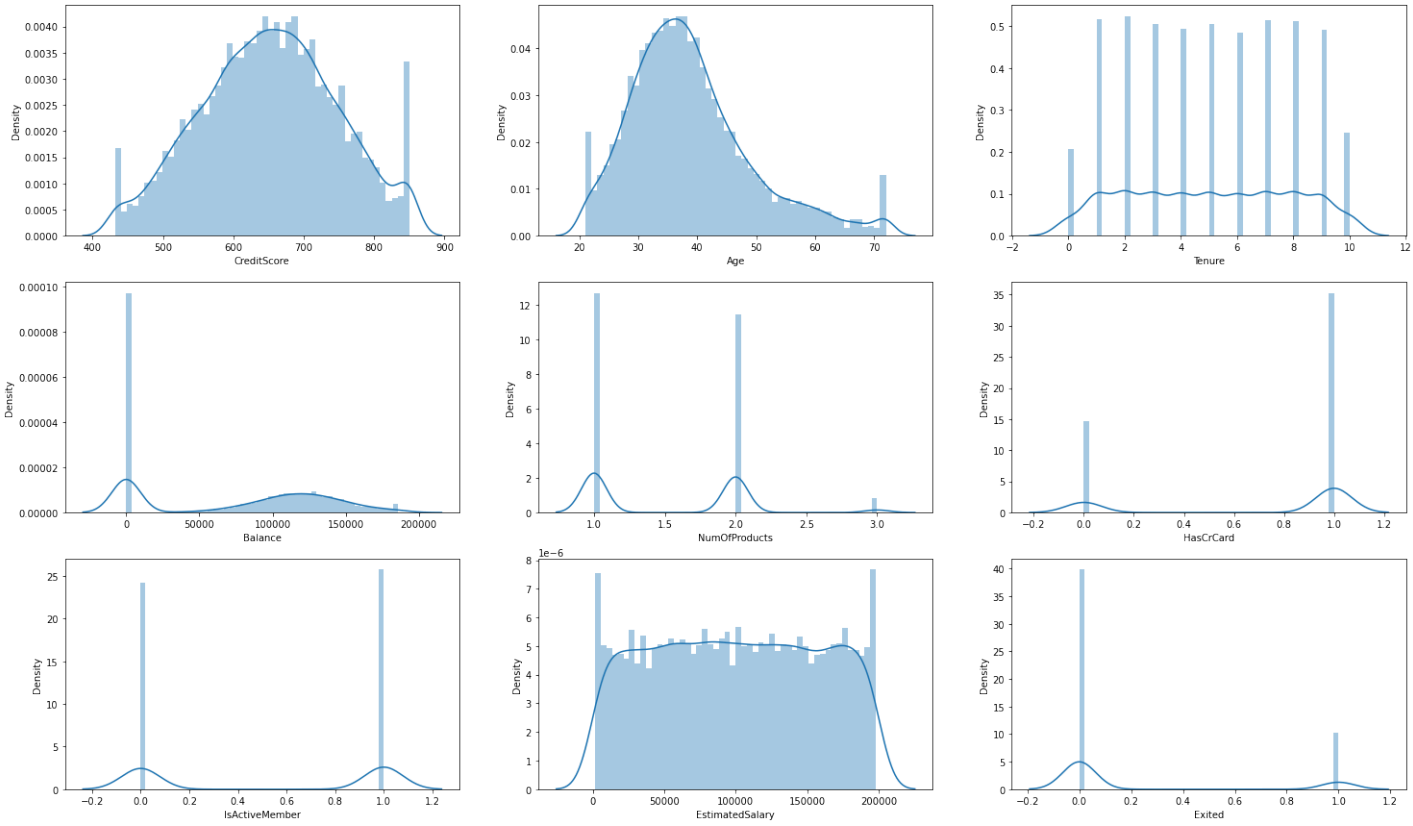
```
# here we use quantile 0.01 as lower limit and 0.99 upper.
df[num_cols]=df[num_cols].clip(lower=df[num_cols].quantile(0.01), upper=df[num_cols].quantile(0.99
df.describe()
```

Out[21]:

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCr
count	10000.00000	1.000000e+04	10000.000000	10000.00000	10000.000000	10000.000000	10000.000000	10000.0
mean	5000.50000	1.569094e+07	650.743200	38.89760	5.012800	76369.720704	1.524200	0.7
std	2886.89568	7.193619e+04	96.115361	10.31522	2.892174	62172.016053	0.560933	0.4
min	1.00000	1.556570e+07	432.000000	21.00000	0.000000	0.000000	1.000000	0.0
25%	2500.75000	1.562853e+07	584.000000	32.00000	3.000000	0.000000	1.000000	0.0
50%	5000.50000	1.569074e+07	652.000000	37.00000	5.000000	97198.540000	1.000000	1.0
75%	7500.25000	1.575323e+07	718.000000	44.00000	7.000000	127644.240000	2.000000	1.0
max	10000.00000	1.581569e+07	850.000000	72.00000	10.000000	185967.985400	3.000000	1.0

In [22]:

```
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
fig, ax = plt.subplots(3, 3, figsize=(25, 15))

for i in range(ax.shape[0]):
    for j in range(ax.shape[1]):
        sns.distplot(df[df[num_cols].columns[i*3+j]], ax=ax[i][j],bins=50)
```

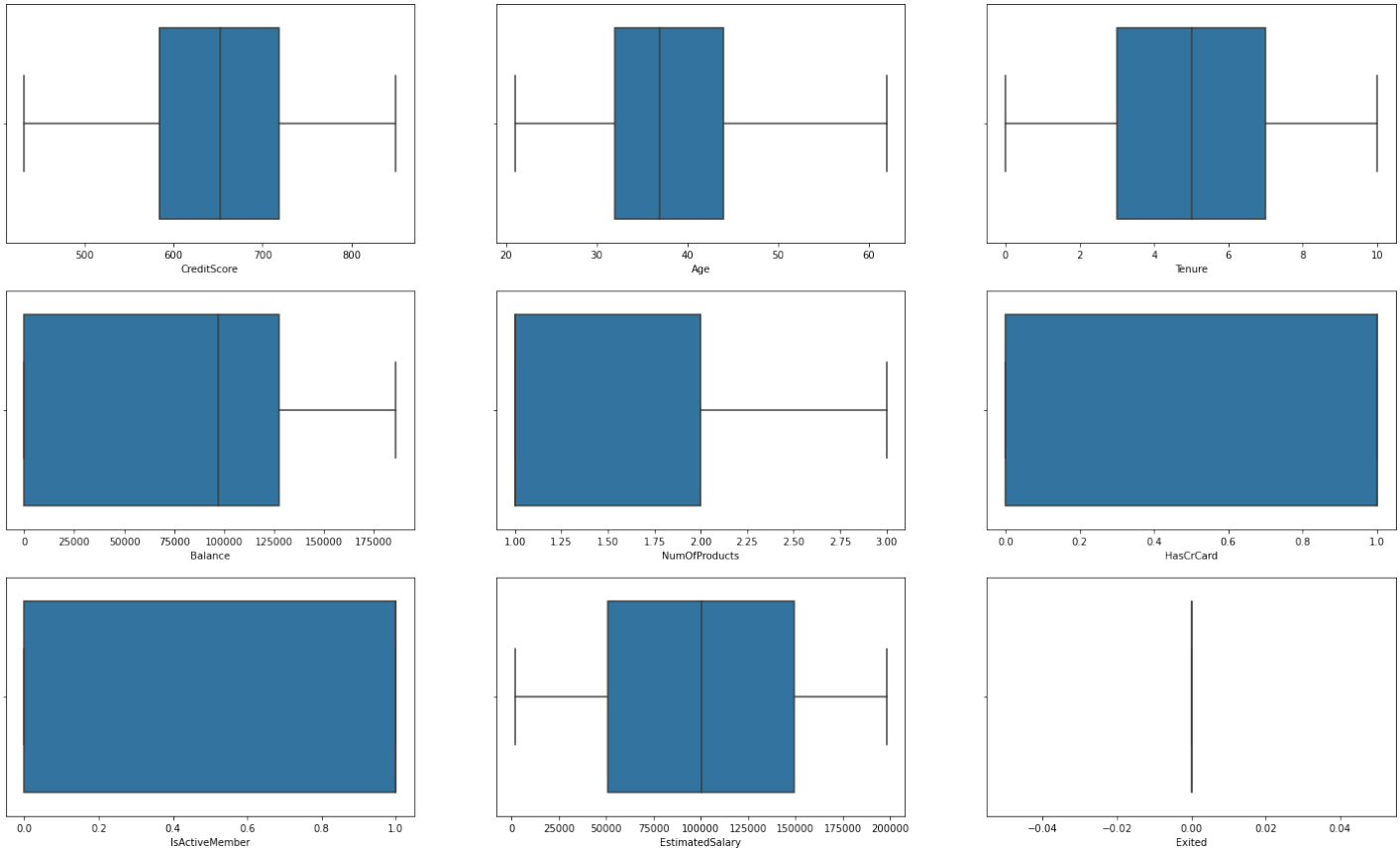


Boxplot the data

Boxplot shows the shape of the distribution, its central value, and its variability

In [23]:

```
import warnings
warnings.simplefilter(action='ignore', category=UserWarning)

fig, ax = plt.subplots(3, 3, figsize=(25, 15))

#plot the features except LOCATION_ID and Risk
for i in range(ax.shape[0]):
    for j in range(ax.shape[1]):
        sns.boxplot(df[df[num_cols].columns[i*3+j]], ax=ax[i][j],orient='v',showfliers=False)
```



Explain the Boxplots:

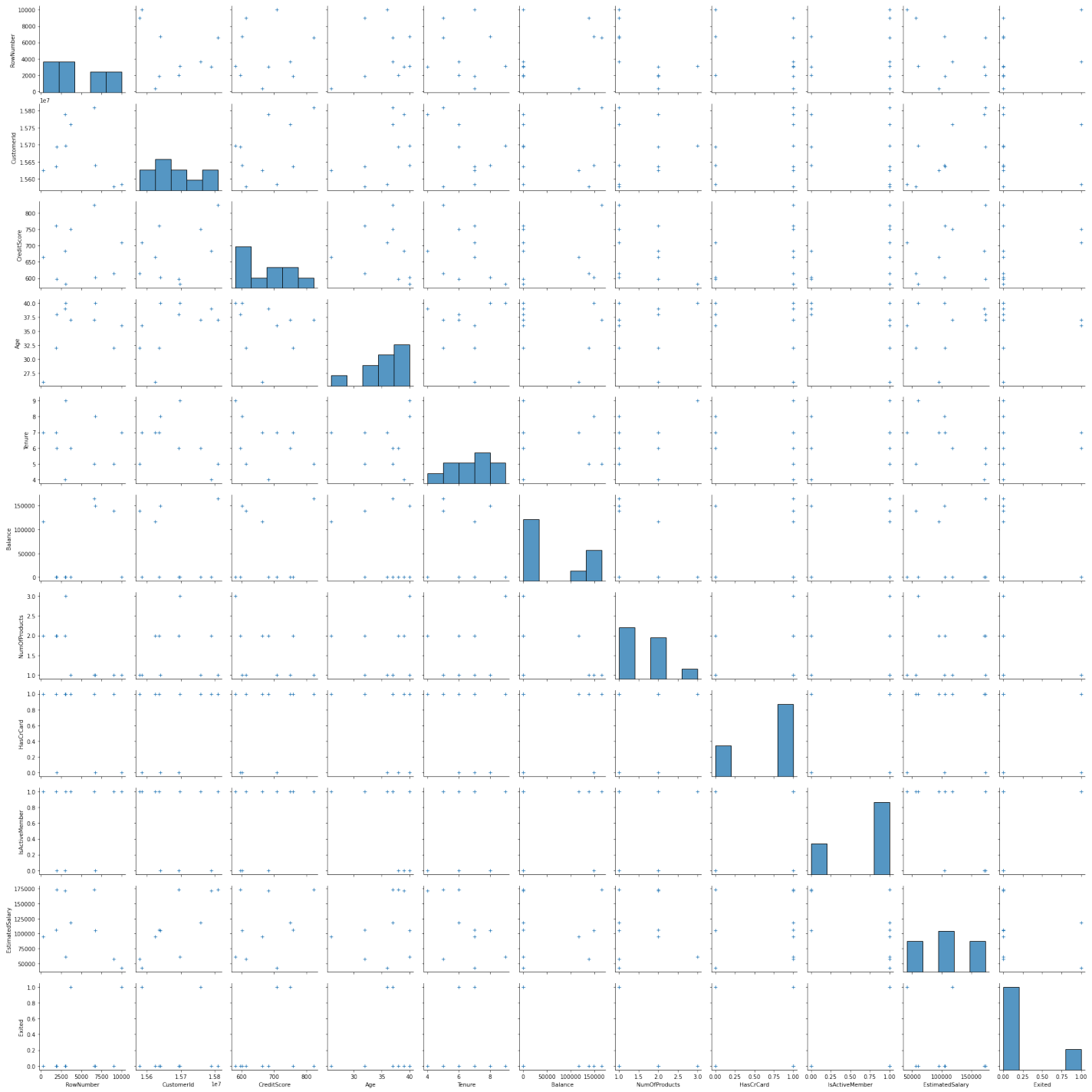
The boxplots tell similar story as the Histograms. None of the distributions seem normal.

Correlation heatmap

Correlation heatmap allows us to see relations between features/attributes. The higher the absolute coefficient, the stronger the correlation is.

In [24]:
```
import seaborn as sns
# df=df.query('`Loan ID`.notnull()', engine='python')
sns.pairplot(df.sample(frac=0.001, replace=True).reset_index(drop=True),plot_kws=dict(marker="+",
```

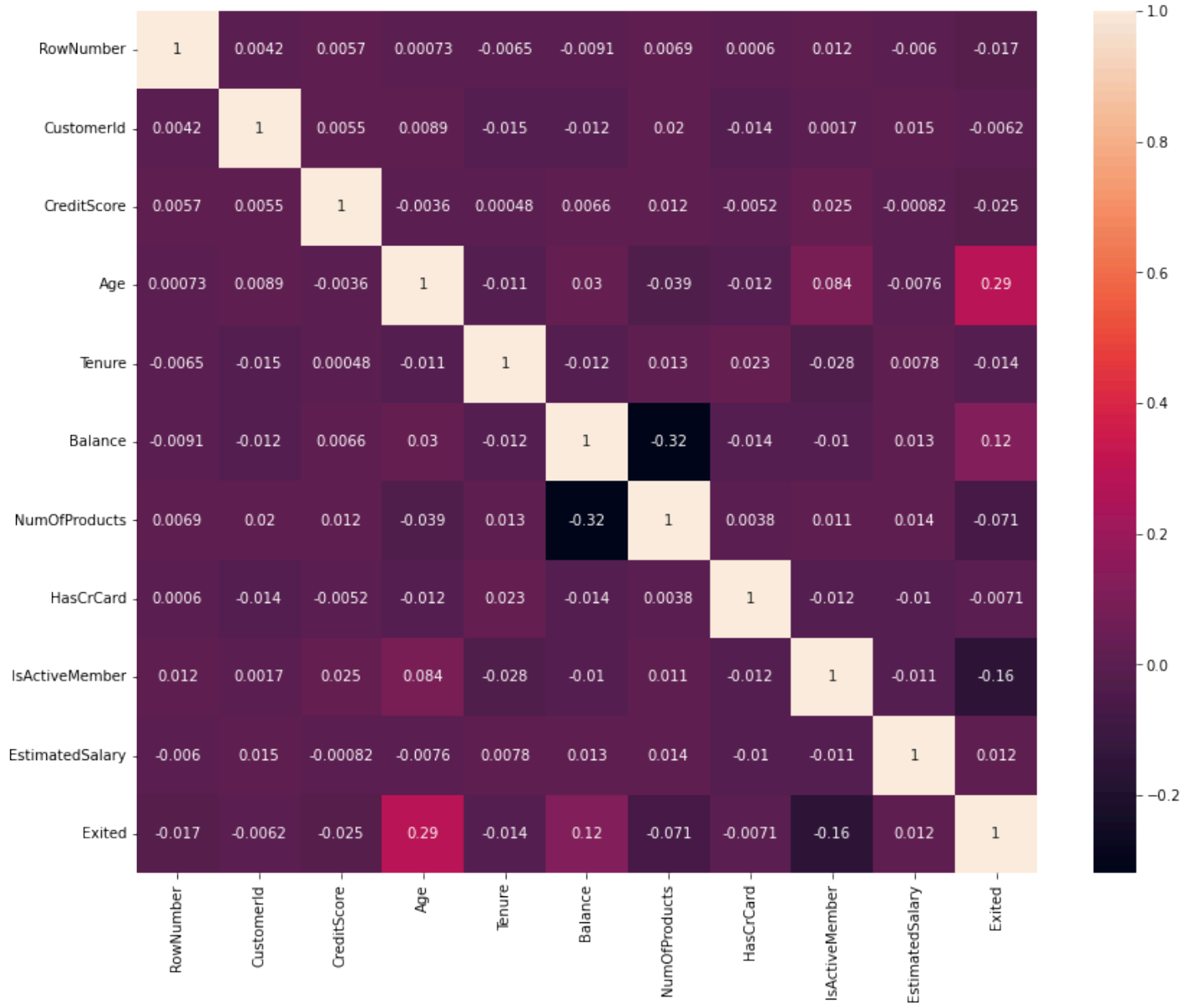
Out[24]:
```
<seaborn.axisgrid.PairGrid at 0x1c9e314e4c0>
```



Numerical vs Numerical

In [25]:

```
# df['Loan Status encoded']=np.where(df['Loan Status']=='Fully Paid',0,1)
# corrMatrix = df.corr()
# plt.figure(figsize = (12,9))
# ax=(sns.heatmap(corrMatrix, annot=True))
# plt.show()

fig, ax = plt.subplots(figsize=(14,11))

ax=(sns.heatmap(df.corr(), annot=True))
```



chisquare test https://www.youtube.com/watch?v=misMgRRV3jQ&ab_channel=MathMeeting

Categorical vs Categorical

In [26]:

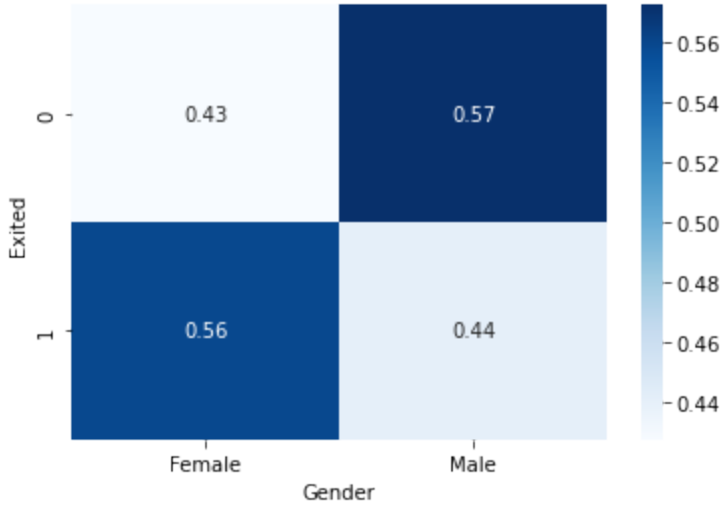
```
from scipy.stats import chisquare,chi2_contingency

v1=df['Exited']

# v2=df['Geography']
v2=df['Gender']
# g, p, dof, expctd=chi2_contingency(pd.crosstab(v1, v2))
print('Chi Square p value =' , chi2_contingency(pd.crosstab(v1, v2))[1])

display(pd.crosstab(v1, v2))
display(pd.crosstab(v1, v2,normalize='index'))
# sns.heatmap(pd.crosstab(v1, v2), annot=True)
sns.heatmap(pd.crosstab(v1, v2,normalize='index'), annot=True,cmap="Blues")
```

```
Chi Square p value = 2.2482100097131755e-26
```

Gender	Female	Male
Exited		
0	3404	4559
1	1139	898

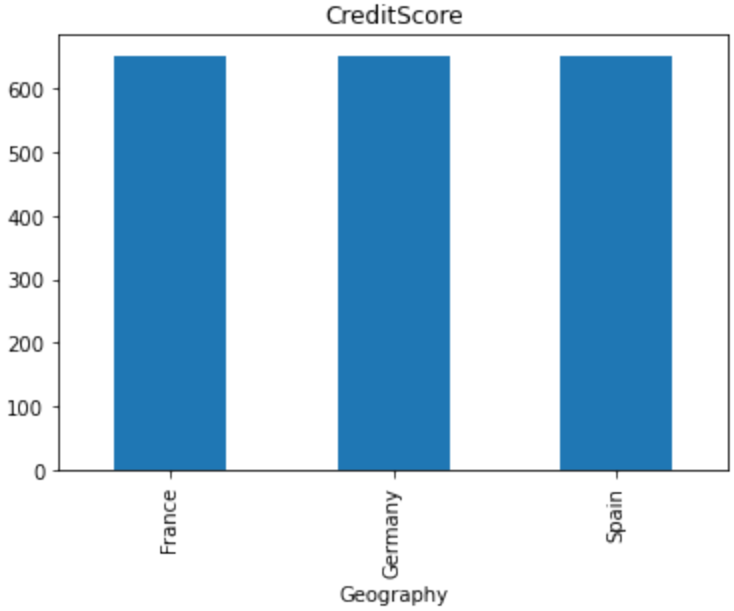
Gender	Female	Male
Exited		
0	0.427477	0.572523
1	0.559156	0.440844

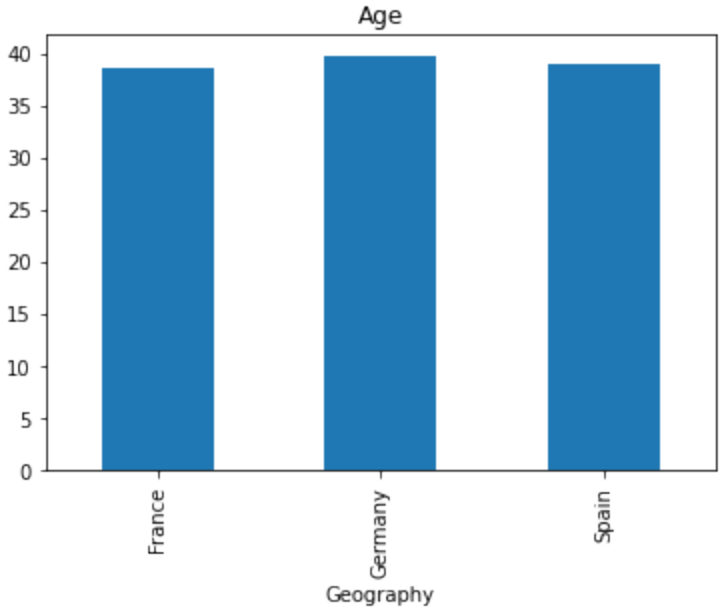
Out[26]:
```
<AxesSubplot:xlabel='Gender', ylabel='Exited'>
```

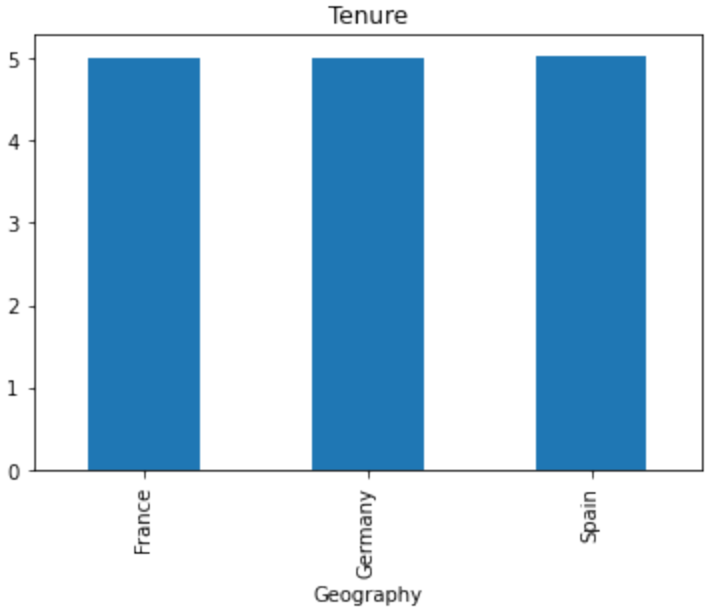


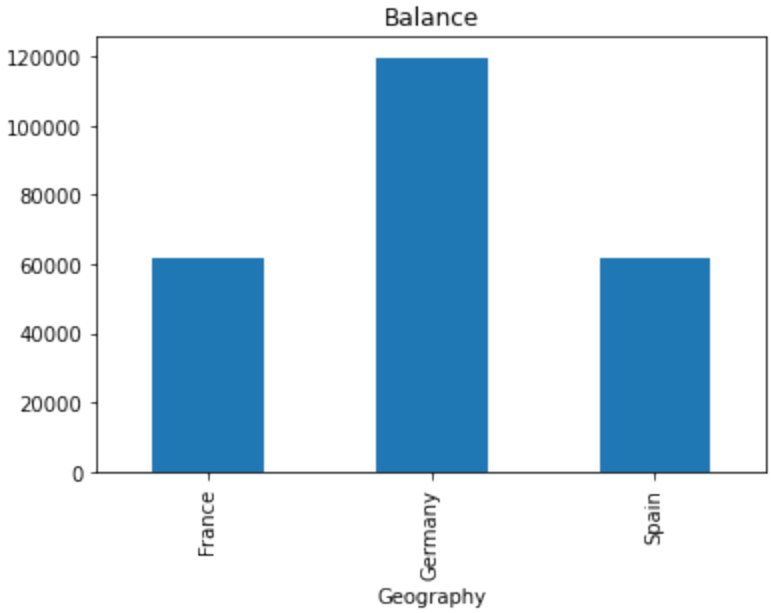
Categorical vs Numerical

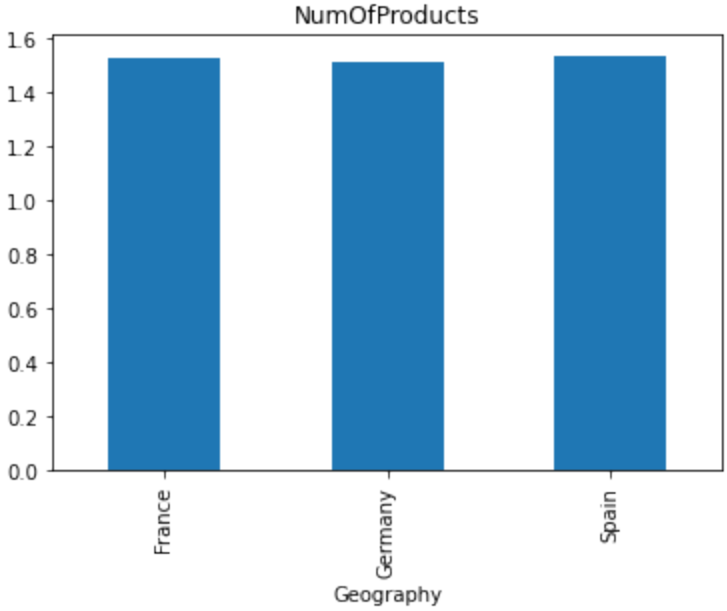
In [27]:
```
for cat_col in cat_cols:
    for num_col in num_cols:
        df.groupby(cat_col)[num_col].mean().plot(kind='bar',title=num_col)
        plt.show()
```

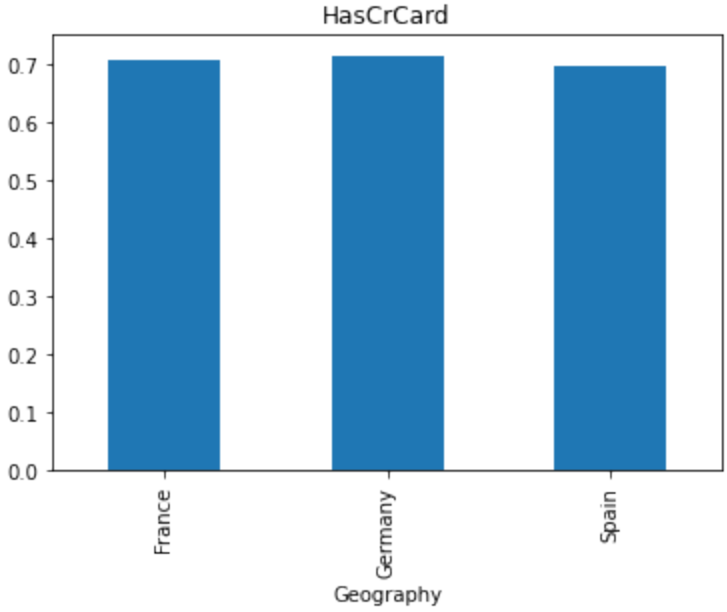

CreditScore
600
500
400
300
200
100
0
France
Germany
Spain
Geography

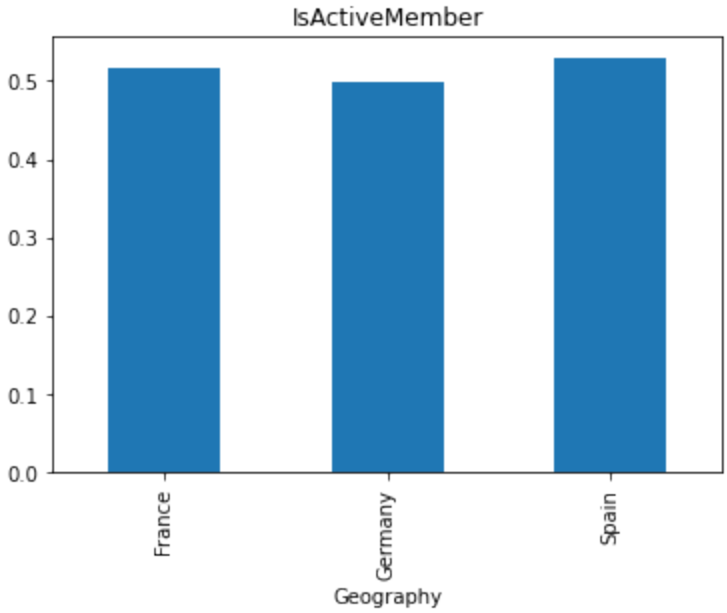

Age
40
35
30
25
20
15
10
5
0
France
Germany
Spain
Geography

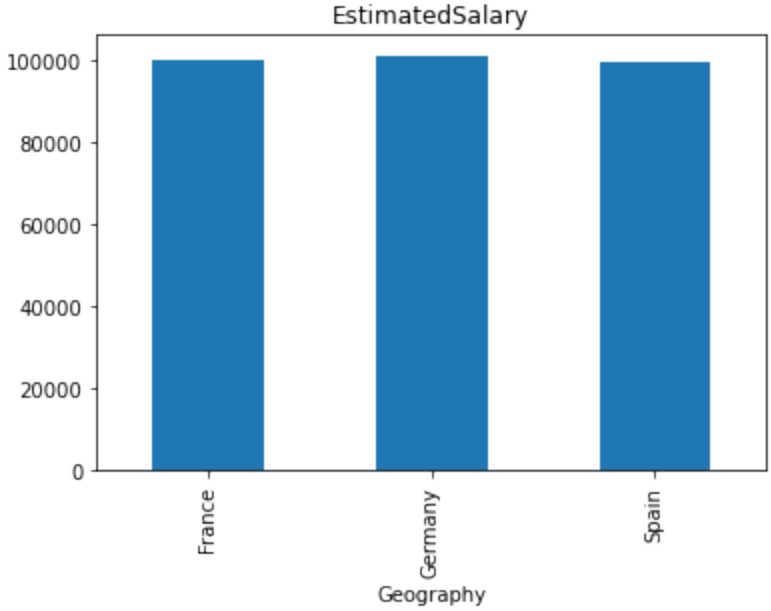

Tenure
5
4
3
2
1
0
France
Germany
Spain
Geography

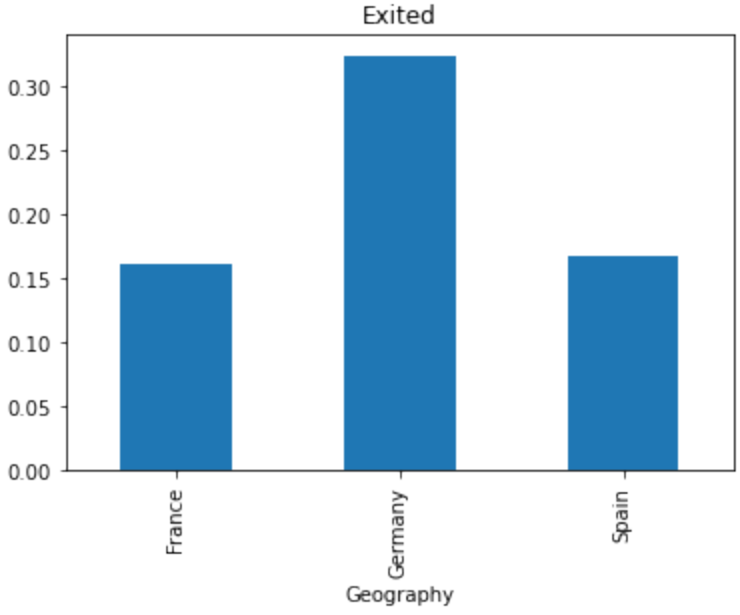

Balance
120000
100000
80000
60000
40000
20000
0
France
Germany
Spain
Geography

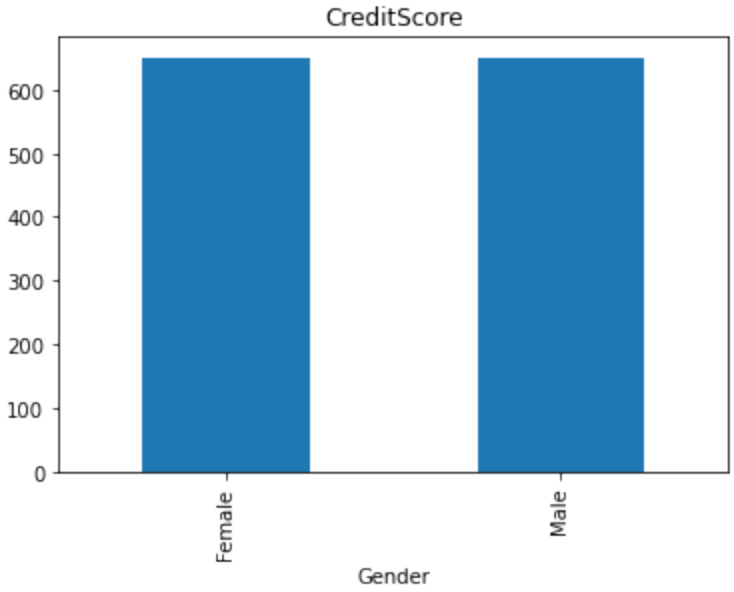

NumOfProducts
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
France
Germany
Spain
Geography

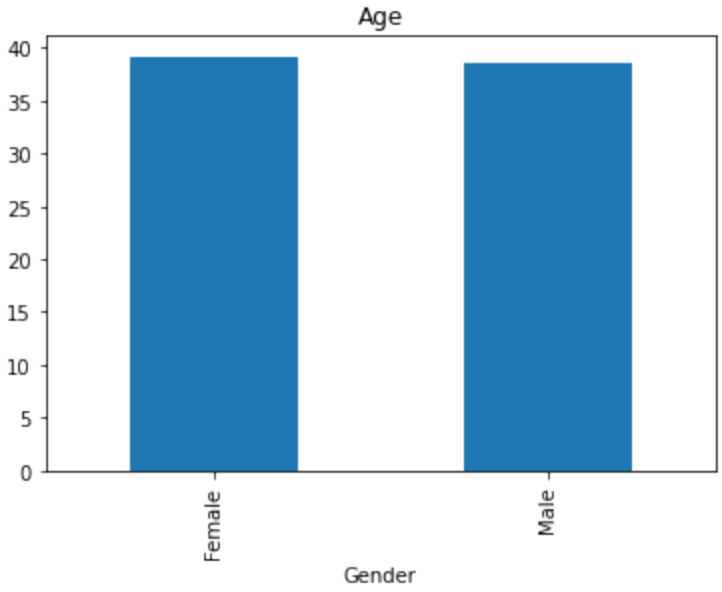

HasCrCard
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
France
Germany
Spain
Geography

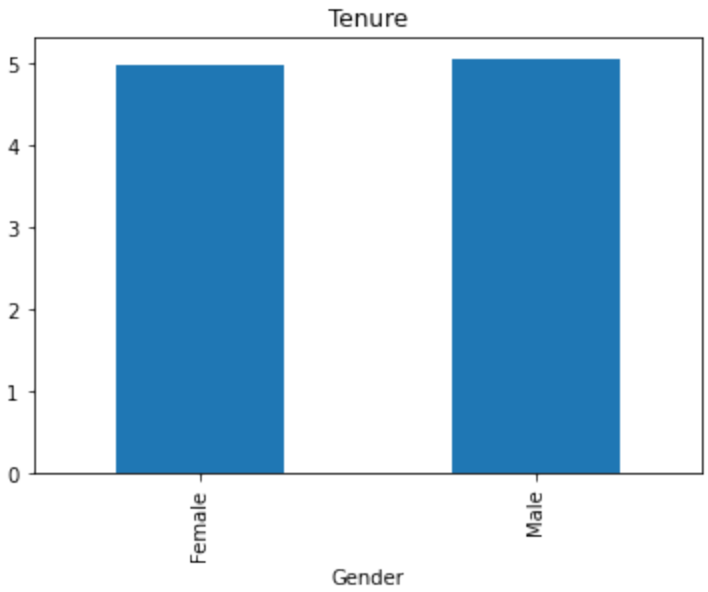

IsActiveMember
0.5
0.4
0.3
0.2
0.1
0.0
France
Germany
Spain
Geography

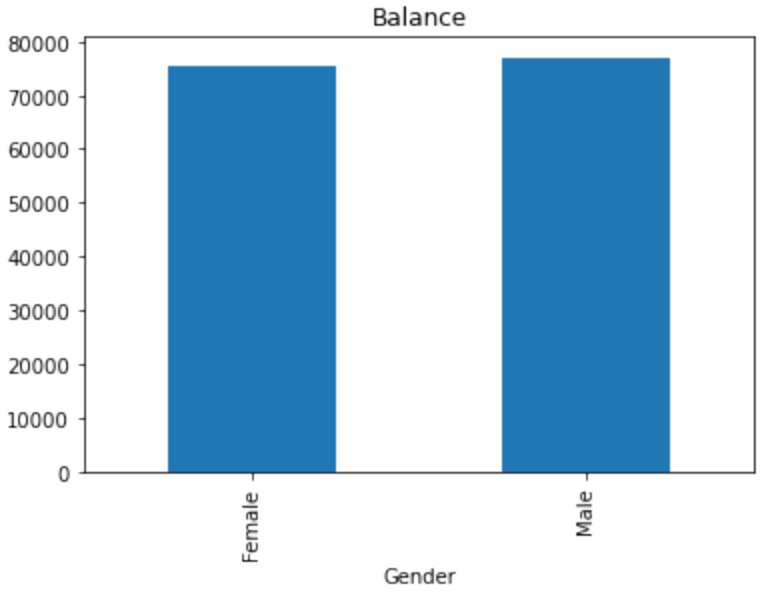

EstimatedSalary
100000
80000
60000
40000
20000
0
France
Germany
Spain
Geography

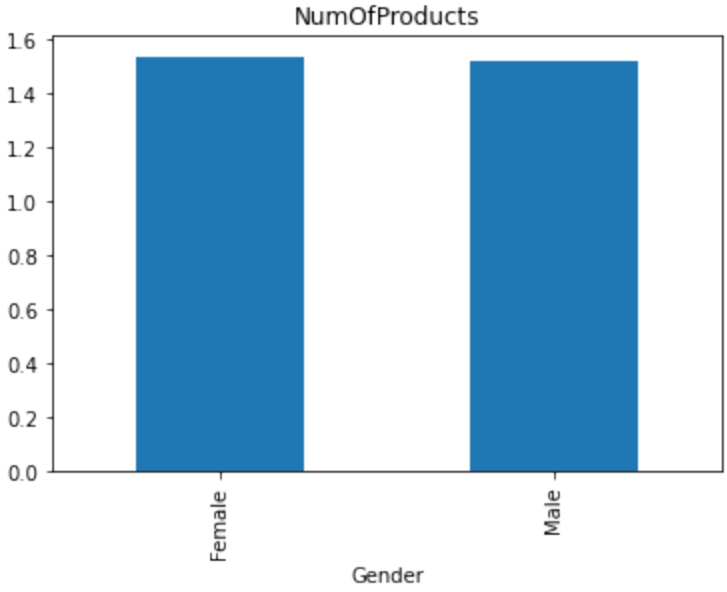

Exited
0.30
0.25
0.20
0.15
0.10
0.05
0.00
France
Germany
Spain
Geography

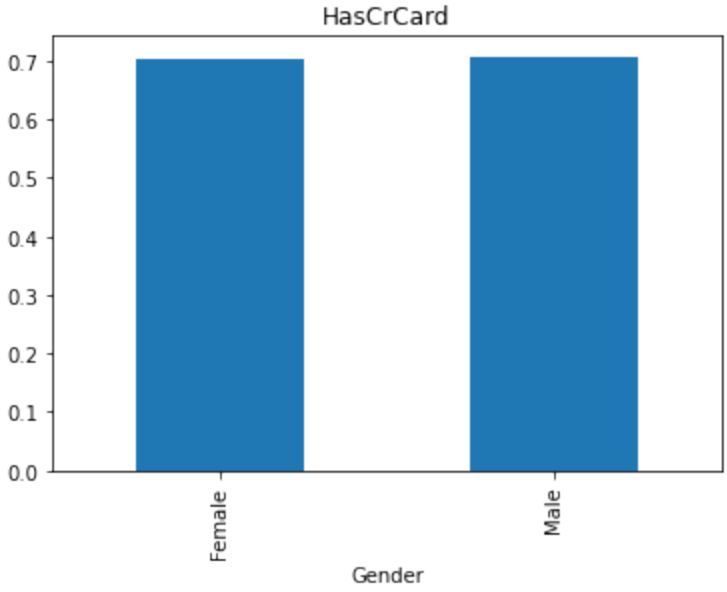

CreditScore
600
500
400
300
200
100
0
Female
Male
Gender

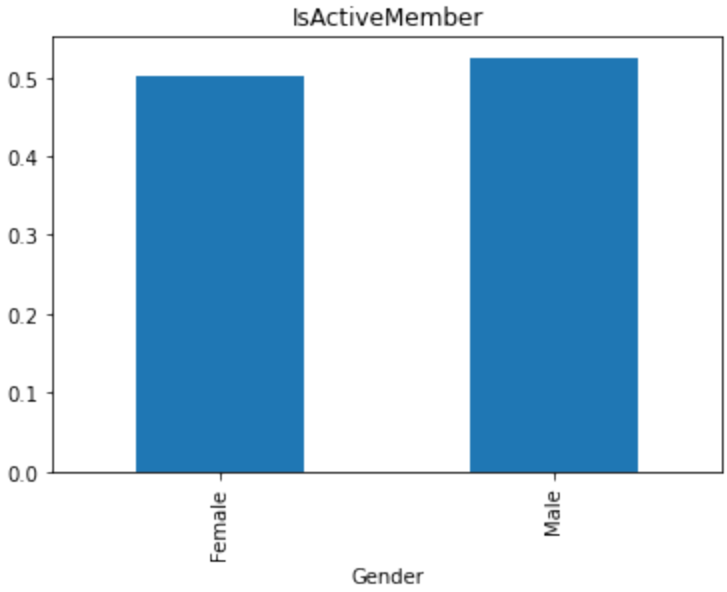

Age
40
35
30
25
20
15
10
5
0
Female
Male
Gender

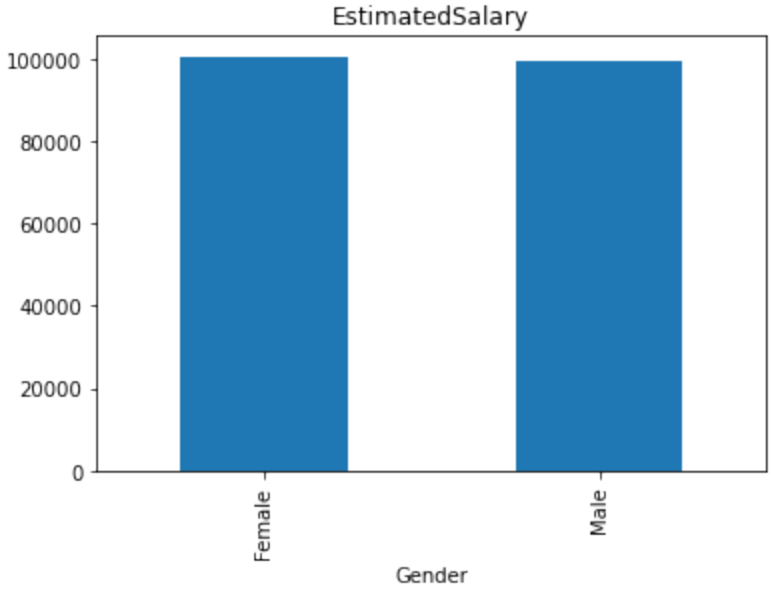

Tenure
5
4
3
2
1
0
Female
Male
Gender

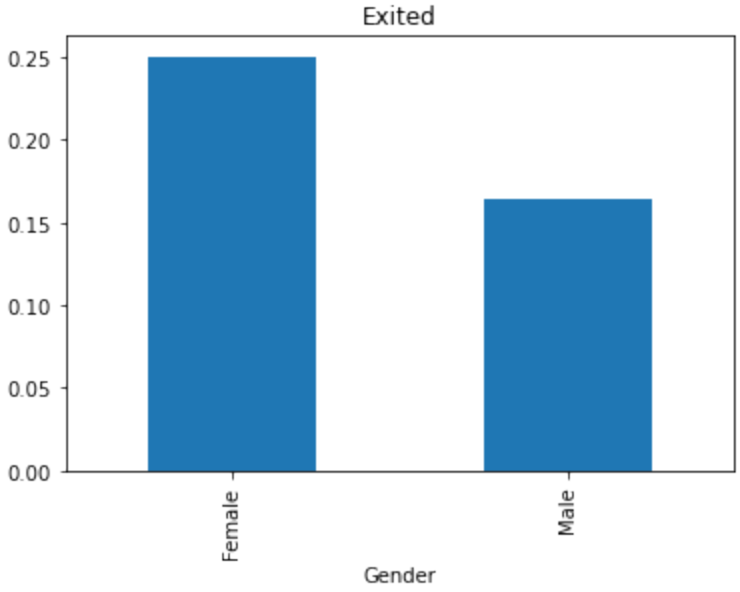

Balance
80000
70000
60000
50000
40000
30000
20000
10000
0
Female
Male
Gender


NumOfProducts
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Female
Male
Gender


HasCrCard
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Female
Male
Gender



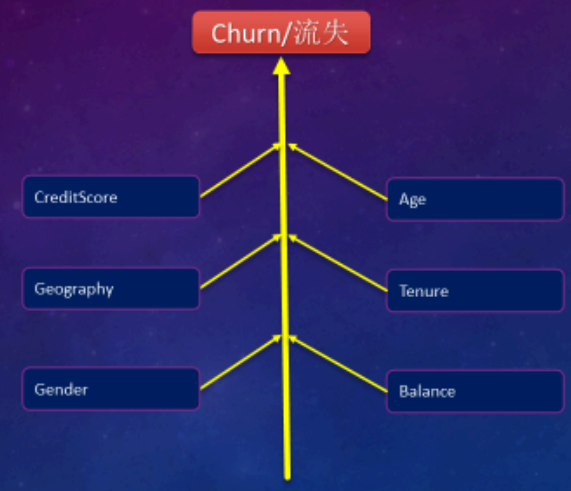




Logestic Regression/Modeling

CHURN ANALYSIS---REGRESSION/回归

Quantify/量化
One to Many/一对多关系
Ranking/排序

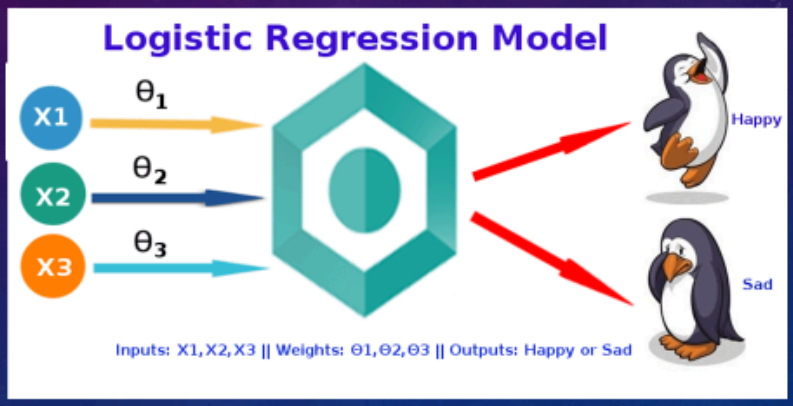


Explain/解释
Control/控制
Predict/预测

Regression Model 回归分析模型

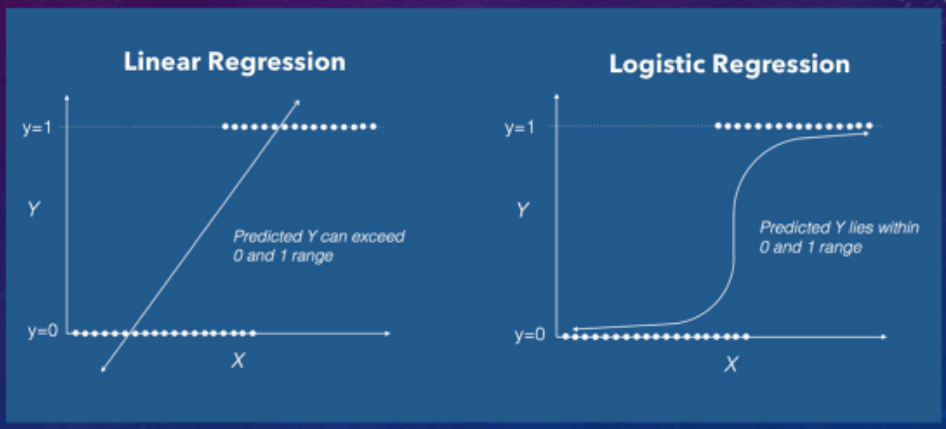
CHURN ANALYSIS

REGRESSION/回归



CHURN ANALYSIS

REGRESSION/回归



CHURN ANALYSIS

Quantify/量化
One to Many/一对多关系
Ranking/排序

Experience or Rules or Models:

If gender is female and age is young then Purchase Probability is high

If gender is male and age is middle-aged then Purchase Probability is low

Gender	Age	Guessed Purchase Probability	What happened
Male	Middle-aged	low	No buy
Female	Young	high	buy

Learning/training

Gender	Age	Guessed Probability	What happened
Male	Middle-aged	90%--buy	No buy
Female	Young	10%--no buy	buy

training

Error

Gender	Age	Guessed Probability	What happened
Male	Middle-aged	10%--no buy	No buy
Female	Young	90%--buy	buy

Error minimized

Learning/training

Male=0
Female=1
Middle-aged=0
Young=1
Buy=1
No buy=0

Gender	Age	Guessed Probability	What happened
0	0	1	0
1	1	0	1

training

Error

Gender	Age	Guessed Probability	What happened
0	0	0	0
1	1	1	1

Error minimized

Learning/training

Male=0
Female=1
Middle-aged=0
Young=1
Buy=1
No buy=0

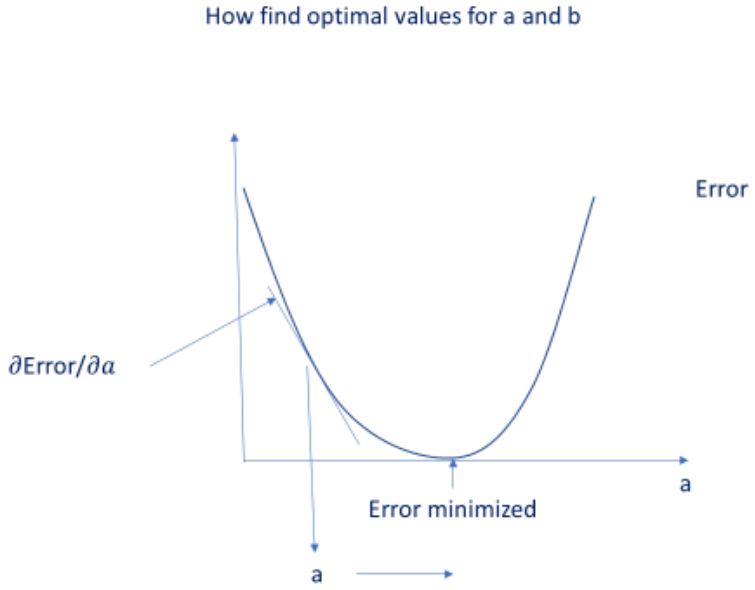
Gender	Parameter 1	Age	Parameter 2	Guessed Probability	What happened
0	a	0	b	=0*a+0*b	0
1	a	1	b	=1*a+1*b	1

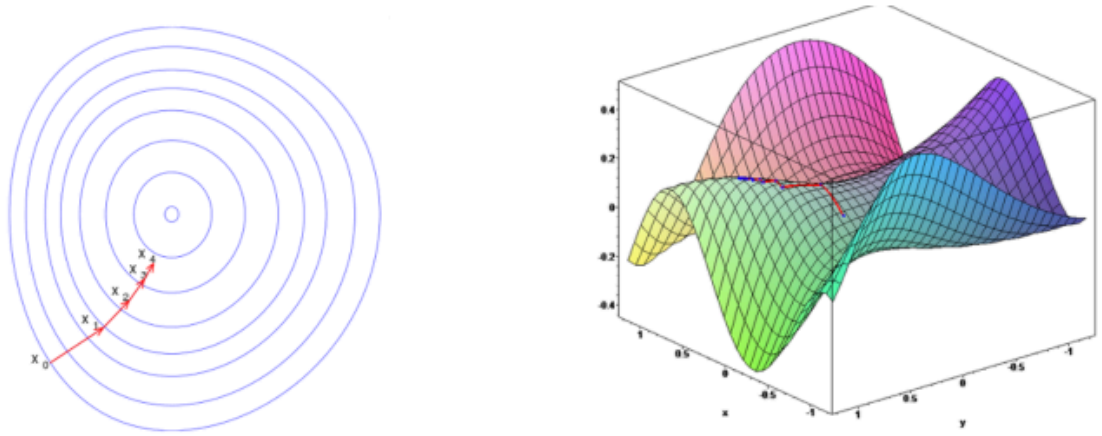
Error

a=1, b=0.6

Gender * a +Age * b=Guessed Probability

Model or Rule





下山大法
Gradient Descent

Data processing and cleaning

In [28]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

In [29]:
```
import pyodbc
import urllib
import sqlalchemy

'''connect to datahub'''

params_datahub = urllib.parse.quote_plus("DRIVER={SQL Server Native Client 11.0};"
                                         "SERVER=localhost\SQLEXPRESS;"
                                         "DATABASE=datahub;"
                                         "UID=sa;"
                                         "PWD=user1")

engine_datahub = sqlalchemy.create_engine("mssql+pyodbc:///?odbc_connect={}".format(params_datahub
```

In [30]:
```
df=pd.read_sql_table(r"marketing_churn",engine_datahub)
df.head()
```

Out[30]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	H
0	7	15592531	Bartlett	822	France	Male	50	7	0.0	2	
1	21	15577657	McDonald	732	France	Male	41	8	0.0	2	
2	77	15614049	Hu	664	France	Male	55	8	0.0	2	
3	94	15640635	Capon	769	France	Male	29	8	0.0	2	
4	142	15724944	Tien	663	France	Male	34	7	0.0	2	

Numerical Variables

In [31]:
```
droplist='Surname'
cat_cols=df.select_dtypes(object).drop(droplist,axis=1).columns.tolist()

droplist=['RowNumber','CustomerId']
num_cols=df.select_dtypes('number').drop(droplist,axis=1).columns.tolist()

df[num_cols]=df[num_cols].clip(lower=df[num_cols].quantile(0.01), upper=df[num_cols].quantile(0.99

# df=df.query('`Attrition`.notnull()',engine='python')
```

In [32]:
```
#replace mnissing value with median, a better representation of the center of the data if it's not

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='median')
for col in num_cols:
    df[col]=imputer.fit_transform(df[col].values.reshape(-1, 1))
```

Categorical Variables

In [33]:
```
imputer = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
for col in cat_cols:
    df[col]=imputer.fit_transform(df[col].values.reshape(-1, 1))
```

In [34]:

```
#encode the attribute
def one_hot(df, cols):
    """
    @param df pandas DataFrame
    @param cols a list of columns to encode
    @return a DataFrame with one-hot encoding
    """
    for each in cols:
        dummies = pd.get_dummies(df[each], prefix=each, drop_first=False)
        df = pd.concat([df, dummies], axis=1)
        df = df.drop([each], axis=1)
    return df


# cat_cols.remove('Attrition')
df=one_hot(df,cat_cols)
df.head()
```

Out[34]:

	RowNumber	CustomerId	Surname	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMer
0	7	15592531	Bartlett	822.0	50.0	7.0	0.0	2.0	1.0	
1	21	15577657	McDonald	732.0	41.0	8.0	0.0	2.0	1.0	
2	77	15614049	Hu	664.0	55.0	8.0	0.0	2.0	1.0	
3	94	15640635	Capon	769.0	29.0	8.0	0.0	2.0	1.0	
4	142	15724944	Tien	663.0	34.0	7.0	0.0	2.0	1.0	

In [35]:

```
x_list = df.columns.tolist()
x_list = [e for e in x_list if e not in ('Exited','Surname','RowNumber','CustomerId')]
df[x_list]
```

Out[35]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geograph
0	822.0	50.0	7.0	0.00	2.0	1.0	1.0	10062.80	
1	732.0	41.0	8.0	0.00	2.0	1.0	1.0	170886.17	
2	664.0	55.0	8.0	0.00	2.0	1.0	1.0	139161.64	
3	769.0	29.0	8.0	0.00	2.0	1.0	1.0	172290.61	
4	663.0	34.0	7.0	0.00	2.0	1.0	1.0	180427.24	
...	...	...	...	...	...	...	...	...	
9995	750.0	38.0	5.0	151532.40	1.0	1.0	1.0	46555.15	
9996	670.0	33.0	8.0	126679.69	1.0	1.0	1.0	39451.09	
9997	739.0	58.0	2.0	101579.28	1.0	1.0	1.0	72168.53	
9998	623.0	48.0	5.0	118469.38	1.0	1.0	1.0	158590.25	
9999	516.0	35.0	10.0	57369.61	1.0	1.0	1.0	101699.77	

10000 rows × 13 columns

In [36]:
```
#For better performance use MinMaxScaler to scale and translates each feature individually such th
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_x =pd.DataFrame(scaler.fit_transform(df[x_list]),columns=x_list)
```

In [37]:
```
#double check the data to see if there is any missing values and all categorical attributes have b
df_x
```

Out[37]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geogr
0	0.933014	0.568627	0.7	0.000000	0.5	1.0	1.0	0.041890	
1	0.717703	0.392157	0.8	0.000000	0.5	1.0	1.0	0.861469	
2	0.555024	0.666667	0.8	0.000000	0.5	1.0	1.0	0.699796	
3	0.806220	0.156863	0.8	0.000000	0.5	1.0	1.0	0.868626	
4	0.552632	0.254902	0.7	0.000000	0.5	1.0	1.0	0.910091	
...	...	...	...	...	...	...	...	...	
9995	0.760766	0.333333	0.5	0.814831	0.0	1.0	1.0	0.227860	
9996	0.569378	0.235294	0.8	0.681191	0.0	1.0	1.0	0.191657	
9997	0.734450	0.725490	0.2	0.546219	0.0	1.0	1.0	0.358390	
9998	0.456938	0.529412	0.5	0.637042	0.0	1.0	1.0	0.798807	
9999	0.200957	0.274510	1.0	0.308492	0.0	1.0	1.0	0.508885	

10000 rows × 13 columns

Create the function to train the model, test it, and visualize the results

In [38]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.metrics import roc_auc_score, roc_curve, f1_score, precision_score, recall_score
from sklearn.metrics import precision_recall_curve, average_precision_score



#splitting the principal training dataset to subtrain and subtest datasets
x_train, x_test, y_train, y_test = train_test_split(df_x, df['Exited'], test_size = .3)

from sklearn.linear_model import LogisticRegression
logit = LogisticRegression()

logit.fit(x_train, y_train)
predictions = logit.predict(x_test)
probabilities = logit.predict_proba(x_test)
```

```
print('Algorithm:', type(logit).__name__)
print("\nClassification report:\n", classification_report(y_test, predictions))
print("Accuracy Score:", accuracy_score(y_test, predictions))
```

```
Algorithm: LogisticRegression

Classification report:
               precision    recall  f1-score   support

         0.0       0.83      0.96      0.89      2411
         1.0       0.56      0.20      0.30       589

    accuracy                           0.81      3000
   macro avg       0.70      0.58      0.60      3000
weighted avg       0.78      0.81      0.78      3000

Accuracy Score: 0.8126666666666666
```

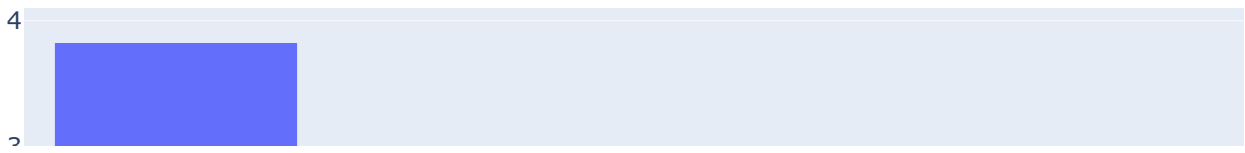
In [39]:

```
#confusion matrix
import plotly.graph_objs as go
from plotly.subplots import make_subplots
import plotly.offline as py
conf_matrix = confusion_matrix(y_test, predictions)

trace=go.Heatmap(z = conf_matrix,x = ["0", "1"],y = ["0", "1"],showscale = False, colorscale = "Pi
fig = make_subplots()
fig.add_trace(trace)
py.iplot(fig)
```

In [40]:

```
column_df = pd.DataFrame(x_train.columns.tolist())
coefficients = pd.DataFrame(logit.coef_.ravel())
coef_sumry = (pd.merge(coefficients, column_df, left_index=True,
                               right_index=True, how="left"))
coef_sumry.columns = ["coefficients", "features"]
coef_sumry = coef_sumry.sort_values(by = "coefficients", ascending=False)
display(coef_sumry)
trace = go.Bar(x = coef_sumry["features"], y = coef_sumry["coefficients"])


fig = make_subplots()
fig.add_trace(trace)
py.iplot(fig)
```

	coefficients	features
1	3.827648	Age
9	0.500111	Geography_Germany
3	0.451504	Balance
11	0.265895	Gender_Female
7	0.232798	EstimatedSalary
5	-0.093226	HasCrCard
10	-0.188593	Geography_Spain
2	-0.218120	Tenure
12	-0.265880	Gender_Male
0	-0.299948	CreditScore
8	-0.311504	Geography_France
4	-0.490580	NumOfProducts
6	-1.088725	IsActiveMember



Feed the parameters to the function created above

Split the data into train dataset and test dataset and use the Hyper Parameters obtained above to generate a Logistic Regression instance and execute the function.

In [41]:
```
help(LogisticRegression())
```

```
Help on LogisticRegression in module sklearn.linear_model._logistic object:

class LogisticRegression(sklearn.linear_model._base.LinearClassifierMixin, sklearn.linear_model._b
ase.SparseCoefMixin, sklearn.base.BaseEstimator)
 |  LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, interce
pt_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='aut
o', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
 |
 |  Logistic Regression (aka logit, MaxEnt) classifier.
 |
 |  In the multiclass case, the training algorithm uses the one-vs-rest (OvR)
 |  scheme if the 'multi_class' option is set to 'ovr', and uses the
 |  cross-entropy loss if the 'multi_class' option is set to 'multinomial'.
 |  (Currently the 'multinomial' option is supported only by the 'lbfgs',
 |  'sag', 'saga' and 'newton-cg' solvers.)
 |
 |  This class implements regularized logistic regression using the
 |  'liblinear' library, 'newton-cg', 'sag', 'saga' and 'lbfgs' solvers. **Note
 |  that regularization is applied by default**. It can handle both dense
 |  and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit
 |  floats for optimal performance; any other input format will be converted
 |  (and copied).
 |
 |  The 'newton-cg', 'sag', and 'lbfgs' solvers support only L2 regularization
 |  with primal formulation, or no regularization. The 'liblinear' solver
 |  supports both L1 and L2 regularization, with a dual formulation only for
 |  the L2 penalty. The Elastic-Net regularization is only supported by the
 |  'saga' solver.
 |
 |  Read more in the :ref:`User Guide <logistic_regression>`.
 |
 |  Parameters
 |  ----------
 |  penalty : {'l1', 'l2', 'elasticnet', 'none'}, default='l2'
 |      Used to specify the norm used in the penalization. The 'newton-cg',
 |      'sag' and 'lbfgs' solvers support only l2 penalties. 'elasticnet' is
 |      only supported by the 'saga' solver. If 'none' (not supported by the
 |      liblinear solver), no regularization is applied.
 |
 |      .. versionadded:: 0.19
 |         l1 penalty with SAGA solver (allowing 'multinomial' + L1)
```

```
|
|  dual : bool, default=False
|      Dual or primal formulation. Dual formulation is only implemented for
|      l2 penalty with liblinear solver. Prefer dual=False when
|      n_samples > n_features.
|
|  tol : float, default=1e-4
|      Tolerance for stopping criteria.
|
|  C : float, default=1.0
|      Inverse of regularization strength; must be a positive float.
|      Like in support vector machines, smaller values specify stronger
|      regularization.
|
|  fit_intercept : bool, default=True
|      Specifies if a constant (a.k.a. bias or intercept) should be
|      added to the decision function.
|
|  intercept_scaling : float, default=1
|      Useful only when the solver 'liblinear' is used
|      and self.fit_intercept is set to True. In this case, x becomes
|      [x, self.intercept_scaling],
|      i.e. a "synthetic" feature with constant value equal to
|      intercept_scaling is appended to the instance vector.
|      The intercept becomes ``intercept_scaling * synthetic_feature_weight``.
|
|      Note! the synthetic feature weight is subject to l1/l2 regularization
|      as all other features.
|      To lessen the effect of regularization on synthetic feature weight
|      (and therefore on the intercept) intercept_scaling has to be increased.
|
|  class_weight : dict or 'balanced', default=None
|      Weights associated with classes in the form ``{class_label: weight}``.
|      If not given, all classes are supposed to have weight one.
|
|      The "balanced" mode uses the values of y to automatically adjust
|      weights inversely proportional to class frequencies in the input data
|      as ``n_samples / (n_classes * np.bincount(y))``.
|
|      Note that these weights will be multiplied with sample_weight (passed
|      through the fit method) if sample_weight is specified.
|
|      .. versionadded:: 0.17
|         *class_weight='balanced'*
|
|  random_state : int, RandomState instance, default=None
|      Used when ``solver`` == 'sag', 'saga' or 'liblinear' to shuffle the
|      data. See :term:`Glossary <random_state>` for details.
|
|  solver : {'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'},             default='lbfgs'
|
|      Algorithm to use in the optimization problem.
|
|      - For small datasets, 'liblinear' is a good choice, whereas 'sag' and
|        'saga' are faster for large ones.
|      - For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs'
|        handle multinomial loss; 'liblinear' is limited to one-versus-rest
|        schemes.
|      - 'newton-cg', 'lbfgs', 'sag' and 'saga' handle L2 or no penalty
|      - 'liblinear' and 'saga' also handle L1 penalty
|      - 'saga' also supports 'elasticnet' penalty
|      - 'liblinear' does not support setting ``penalty='none'``
```

```
|
|      Note that 'sag' and 'saga' fast convergence is only guaranteed on
|      features with approximately the same scale. You can
|      preprocess the data with a scaler from sklearn.preprocessing.
|
|      .. versionadded:: 0.17
|         Stochastic Average Gradient descent solver.
|      .. versionadded:: 0.19
|         SAGA solver.
|      .. versionchanged:: 0.22
|          The default solver changed from 'liblinear' to 'lbfgs' in 0.22.
|
|  max_iter : int, default=100
|      Maximum number of iterations taken for the solvers to converge.
|
|  multi_class : {'auto', 'ovr', 'multinomial'}, default='auto'
|      If the option chosen is 'ovr', then a binary problem is fit for each
|      label. For 'multinomial' the loss minimised is the multinomial loss fit
|      across the entire probability distribution, *even when the data is
|      binary*. 'multinomial' is unavailable when solver='liblinear'.
|      'auto' selects 'ovr' if the data is binary, or if solver='liblinear',
|      and otherwise selects 'multinomial'.
|
|      .. versionadded:: 0.18
|         Stochastic Average Gradient descent solver for 'multinomial' case.
|      .. versionchanged:: 0.22
|          Default changed from 'ovr' to 'auto' in 0.22.
|
|  verbose : int, default=0
|      For the liblinear and lbfgs solvers set verbose to any positive
|      number for verbosity.
|
|  warm_start : bool, default=False
|      When set to True, reuse the solution of the previous call to fit as
|      initialization, otherwise, just erase the previous solution.
|      Useless for liblinear solver. See :term:`the Glossary <warm_start>`.
|
|      .. versionadded:: 0.17
|         *warm_start* to support *lbfgs*, *newton-cg*, *sag*, *saga* solvers.
|
|  n_jobs : int, default=None
|      Number of CPU cores used when parallelizing over classes if
|      multi_class='ovr'". This parameter is ignored when the ``solver`` is
|      set to 'liblinear' regardless of whether 'multi_class' is specified or
|      not. ``None`` means 1 unless in a :obj:`joblib.parallel_backend`
|      context. ``-1`` means using all processors.
|      See :term:`Glossary <n_jobs>` for more details.
|
|  l1_ratio : float, default=None
|      The Elastic-Net mixing parameter, with ``0 <= l1_ratio <= 1``. Only
|      used if ``penalty='elasticnet'``. Setting ``l1_ratio=0`` is equivalent
|      to using ``penalty='l2'``, while setting ``l1_ratio=1`` is equivalent
|      to using ``penalty='l1'``. For ``0 < l1_ratio <1``, the penalty is a
|      combination of L1 and L2.
|
|  Attributes
|  ----------
|
|  classes_ : ndarray of shape (n_classes, )
|      A list of class labels known to the classifier.
|
|  coef_ : ndarray of shape (1, n_features) or (n_classes, n_features)
```

```
|      Coefficient of the features in the decision function.
|
|      `coef_` is of shape (1, n_features) when the given problem is binary.
|      In particular, when `multi_class='multinomial'`, `coef_` corresponds
|      to outcome 1 (True) and `-coef_` corresponds to outcome 0 (False).
|
|  intercept_ : ndarray of shape (1,) or (n_classes,)
|      Intercept (a.k.a. bias) added to the decision function.
|
|      If `fit_intercept` is set to False, the intercept is set to zero.
|      `intercept_` is of shape (1,) when the given problem is binary.
|      In particular, when `multi_class='multinomial'`, `intercept_`
|      corresponds to outcome 1 (True) and `-intercept_` corresponds to
|      outcome 0 (False).
|
|  n_iter_ : ndarray of shape (n_classes,) or (1, )
|      Actual number of iterations for all classes. If binary or multinomial,
|      it returns only 1 element. For liblinear solver, only the maximum
|      number of iteration across all classes is given.
|
|      .. versionchanged:: 0.20
|
|          In SciPy <= 1.0.0 the number of lbfgs iterations may exceed
|          ``max_iter``. ``n_iter_`` will now report at most ``max_iter``.
|
|  See Also
|  --------
|  SGDClassifier : Incrementally trained logistic regression (when given
|      the parameter ``loss="log"``).
|  LogisticRegressionCV : Logistic regression with built-in cross validation.
|
|  Notes
|  -----
|  The underlying C implementation uses a random number generator to
|  select features when fitting the model. It is thus not uncommon,
|  to have slightly different results for the same input data. If
|  that happens, try with a smaller tol parameter.
|
|  Predict output may not match that of standalone liblinear in certain
|  cases. See :ref:`differences from liblinear <liblinear_differences>`
|  in the narrative documentation.
|
|  References
|  ----------
|
|  L-BFGS-B -- Software for Large-scale Bound-constrained Optimization
|      Ciyou Zhu, Richard Byrd, Jorge Nocedal and Jose Luis Morales.
|      http://users.iems.northwestern.edu/~nocedal/lbfgsb.html
|
|  LIBLINEAR -- A Library for Large Linear Classification
|      https://www.csie.ntu.edu.tw/~cjlin/liblinear/
|
|  SAG -- Mark Schmidt, Nicolas Le Roux, and Francis Bach
|      Minimizing Finite Sums with the Stochastic Average Gradient
|      https://hal.inria.fr/hal-00860051/document
|
|  SAGA -- Defazio, A., Bach F. & Lacoste-Julien S. (2014).
|      SAGA: A Fast Incremental Gradient Method With Support
|      for Non-Strongly Convex Composite Objectives
|      https://arxiv.org/abs/1407.0202
|
|  Hsiang-Fu Yu, Fang-Lan Huang, Chih-Jen Lin (2011). Dual coordinate descent
```

```
 |      methods for logistic regression and maximum entropy models.
 |      Machine Learning 85(1-2):41-75.
 |      https://www.csie.ntu.edu.tw/~cjlin/papers/maxent_dual.pdf
 |
 |  Examples
 |  --------
 |  >>> from sklearn.datasets import load_iris
 |  >>> from sklearn.linear_model import LogisticRegression
 |  >>> X, y = load_iris(return_X_y=True)
 |  >>> clf = LogisticRegression(random_state=0).fit(X, y)
 |  >>> clf.predict(X[:2, :])
 |  array([0, 0])
 |  >>> clf.predict_proba(X[:2, :])
 |  array([[9.8...e-01, 1.8...e-02, 1.4...e-08],
 |         [9.7...e-01, 2.8...e-02, ...e-08]])
 |  >>> clf.score(X, y)
 |  0.97...
 |
 |  Method resolution order:
 |      LogisticRegression
 |      sklearn.linear_model._base.LinearClassifierMixin
 |      sklearn.base.ClassifierMixin
 |      sklearn.linear_model._base.SparseCoefMixin
 |      sklearn.base.BaseEstimator
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __init__(self, penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_s
caling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto',
verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  fit(self, X, y, sample_weight=None)
 |      Fit the model according to the given training data.
 |
 |      Parameters
 |      ----------
 |      X : {array-like, sparse matrix} of shape (n_samples, n_features)
 |          Training vector, where n_samples is the number of samples and
 |          n_features is the number of features.
 |
 |      y : array-like of shape (n_samples,)
 |          Target vector relative to X.
 |
 |      sample_weight : array-like of shape (n_samples,) default=None
 |          Array of weights that are assigned to individual samples.
 |          If not provided, then each sample is given unit weight.
 |
 |          .. versionadded:: 0.17
 |             *sample_weight* support to LogisticRegression.
 |
 |      Returns
 |      -------
 |      self
 |          Fitted estimator.
 |
 |      Notes
 |      -----
 |      The SAGA solver supports both float64 and float32 bit arrays.
 |
 |  predict_log_proba(self, X)
```

```
|      Predict logarithm of probability estimates.
|
|      The returned estimates for all classes are ordered by the
|      label of classes.
|
|      Parameters
|      ----------
|      X : array-like of shape (n_samples, n_features)
|          Vector to be scored, where `n_samples` is the number of samples and
|          `n_features` is the number of features.
|
|      Returns
|      -------
|      T : array-like of shape (n_samples, n_classes)
|          Returns the log-probability of the sample for each class in the
|          model, where classes are ordered as they are in ``self.classes_``.
|
|  predict_proba(self, X)
|      Probability estimates.
|
|      The returned estimates for all classes are ordered by the
|      label of classes.
|
|      For a multi_class problem, if multi_class is set to be "multinomial"
|      the softmax function is used to find the predicted probability of
|      each class.
|      Else use a one-vs-rest approach, i.e calculate the probability
|      of each class assuming it to be positive using the logistic function.
|      and normalize these values across all the classes.
|
|      Parameters
|      ----------
|      X : array-like of shape (n_samples, n_features)
|          Vector to be scored, where `n_samples` is the number of samples and
|          `n_features` is the number of features.
|
|      Returns
|      -------
|      T : array-like of shape (n_samples, n_classes)
|          Returns the probability of the sample for each class in the model,
|          where classes are ordered as they are in ``self.classes_``.
|
|  ----------------------------------------------------------------------
|  Methods inherited from sklearn.linear_model._base.LinearClassifierMixin:
|
|  decision_function(self, X)
|      Predict confidence scores for samples.
|
|      The confidence score for a sample is proportional to the signed
|      distance of that sample to the hyperplane.
|
|      Parameters
|      ----------
|      X : array-like or sparse matrix, shape (n_samples, n_features)
|          Samples.
|
|      Returns
|      -------
|      array, shape=(n_samples,) if n_classes == 2 else (n_samples, n_classes)
|          Confidence scores per (sample, class) combination. In the binary
|          case, confidence score for self.classes_[1] where >0 means this
|          class would be predicted.
```

```
|
|  predict(self, X)
|      Predict class labels for samples in X.
|
|      Parameters
|      ----------
|      X : array-like or sparse matrix, shape (n_samples, n_features)
|          Samples.
|
|      Returns
|      -------
|      C : array, shape [n_samples]
|          Predicted class label per sample.
|
|  ----------------------------------------------------------------------
|  Methods inherited from sklearn.base.ClassifierMixin:
|
|  score(self, X, y, sample_weight=None)
|      Return the mean accuracy on the given test data and labels.
|
|      In multi-label classification, this is the subset accuracy
|      which is a harsh metric since you require for each sample that
|      each label set be correctly predicted.
|
|      Parameters
|      ----------
|      X : array-like of shape (n_samples, n_features)
|          Test samples.
|
|      y : array-like of shape (n_samples,) or (n_samples, n_outputs)
|          True labels for `X`.
|
|      sample_weight : array-like of shape (n_samples,), default=None
|          Sample weights.
|
|      Returns
|      -------
|      score : float
|          Mean accuracy of ``self.predict(X)`` wrt. `y`.
|
|  ----------------------------------------------------------------------
|  Data descriptors inherited from sklearn.base.ClassifierMixin:
|
|  __dict__
|      dictionary for instance variables (if defined)
|
|  __weakref__
|      list of weak references to the object (if defined)
|
|  ----------------------------------------------------------------------
|  Methods inherited from sklearn.linear_model._base.SparseCoefMixin:
|
|  densify(self)
|      Convert coefficient matrix to dense array format.
|
|      Converts the ``coef_`` member (back) to a numpy.ndarray. This is the
|      default format of ``coef_`` and is required for fitting, so calling
|      this method is only required on models that have previously been
|      sparsified; otherwise, it is a no-op.
|
|      Returns
|      -------
```

```
|      self
|          Fitted estimator.
|
|  sparsify(self)
|      Convert coefficient matrix to sparse format.
|
|      Converts the ``coef_`` member to a scipy.sparse matrix, which for
|      L1-regularized models can be much more memory- and storage-efficient
|      than the usual numpy.ndarray representation.
|
|      The ``intercept_`` member is not converted.
|
|      Returns
|      -------
|      self
|          Fitted estimator.
|
|      Notes
|      -----
|      For non-sparse models, i.e. when there are not many zeros in ``coef_``,
|      this may actually *increase* memory usage, so use this method with
|      care. A rule of thumb is that the number of zero elements, which can
|      be computed with ``(coef_ == 0).sum()``, must be more than 50% for this
|      to provide significant benefits.
|
|      After calling this method, further fitting with the partial_fit
|      method (if any) will not work until you call densify.
|
|  ----------------------------------------------------------------------
|  Methods inherited from sklearn.base.BaseEstimator:
|
|  __getstate__(self)
|
|  __repr__(self, N_CHAR_MAX=700)
|      Return repr(self).
|
|  __setstate__(self, state)
|
|  get_params(self, deep=True)
|      Get parameters for this estimator.
|
|      Parameters
|      ----------
|      deep : bool, default=True
|          If True, will return the parameters for this estimator and
|          contained subobjects that are estimators.
|
|      Returns
|      -------
|      params : dict
|          Parameter names mapped to their values.
|
|  set_params(self, **params)
|      Set the parameters of this estimator.
|
|      The method works on simple estimators as well as on nested objects
|      (such as :class:`~sklearn.pipeline.Pipeline`). The latter have
|      parameters of the form ``<component>__<parameter>`` so that it's
|      possible to update each component of a nested object.
|
|      Parameters
|      ----------
```

```
|      **params : dict
|          Estimator parameters.
|
|      Returns
|      -------
|      self : estimator instance
|          Estimator instance.
```

Interpret the results:

Productization of your Insights/Recommendations

In [1]:
```
df=pd.read_sql_table(r"marketing_churn",engine_datahub)
df.head()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
~\AppData\Local\Temp/ipykernel_37360/312072836.py in <module>
----> 1 df=pd.read_sql_table(r"marketing_churn",engine_datahub)
      2 df.head()

NameError: name 'pd' is not defined
```

In [43]:
```
df['probabily'] = logit.predict_proba(df_x)[:,1]
df.sort_values(by='probabily',ascending=False).head(50)
```

Out[43]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfP
7475	7500	15790113	Schofield	609	Germany	Female	71	6	113317.10	
7672	9588	15653050	Norriss	719	Germany	Female	76	10	95052.29	
7538	9556	15655360	Chikelu	782	Germany	Female	72	5	148666.99	
7613	4816	15737647	Obioma	775	Germany	Female	77	6	135120.56	
6903	7630	15591107	Flemming	723	Germany	Female	68	3	110357.00	
7396	4436	15648967	Ch'en	698	Germany	Female	64	1	169362.43	
7461	7009	15638610	Kennedy	635	Germany	Female	65	5	117325.54	
4858	8489	15794360	Hao	592	Germany	Female	70	5	71816.74	
6870	4560	15668248	Quinn	528	Germany	Female	62	7	133201.17	
7899	8157	15785576	Mayrhofer	434	Germany	Male	71	9	119496.87	
3828	9748	15775761	Iweobiegbunam	610	Germany	Female	69	5	86038.21	
8368	3532	15653251	Hickey	408	France	Female	84	8	87873.39	
7294	618	15766575	Larionova	612	Germany	Female	62	8	140745.33	
6954	4397	15691119	Martin	721	Germany	Male	68	4	136525.99	
7890	7693	15807889	Wood	634	Germany	Male	74	5	108891.70	
7655	8189	15623314	Tucker	506	Germany	Female	59	3	190353.08	
6820	417	15720559	Heath	487	Germany	Female	61	5	110368.03	

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfP
7344	2474	15679249	Chou	351	Germany	Female	57	4	163146.46	
7485	7814	15592751	Okwudiliolisa	684	Germany	Female	63	3	81245.79	
4822	4752	15763256	Sheppard	661	Germany	Female	64	8	128751.65	
3778	4464	15778975	Nnonso	850	Germany	Female	70	1	96947.58	
8415	8099	15594391	Samaniego	770	France	Female	68	2	183555.24	
5410	9293	15677764	Chao	461	Germany	Female	74	1	186445.31	
6905	7734	15596013	Akhtar	694	Germany	Female	58	1	143212.22	
7551	314	15797960	Skinner	806	Germany	Female	59	0	135296.33	
3780	4564	15694376	Sullivan	705	Germany	Female	64	3	153469.26	
7356	2778	15776233	Kruglova	758	Germany	Female	61	8	125397.21	
7531	9333	15638882	Cardell	710	Germany	Female	62	9	148214.36	
6919	9603	15603135	Boni	634	Germany	Female	59	3	95727.05	
3351	4502	15678916	Kelly	512	France	Female	75	2	0.00	
7334	2191	15609998	Okwudilichukwu	700	Germany	Female	59	5	137648.41	
7790	9473	15579345	Murphy	775	Germany	Female	74	0	161371.50	
7288	421	15810418	T'ang	756	Germany	Female	60	3	115924.89	
7423	5577	15635964	Eve	566	Germany	Male	65	4	120100.41	
7246	6582	15598744	Ch'ang	576	Germany	Female	71	6	140273.47	
5297	5440	15582168	Muravyova	713	Germany	Female	61	4	149525.34	
7091	6531	15808851	Bufkin	511	Germany	Female	75	9	105609.17	
8100	400	15646372	Outhwaite	616	France	Female	66	1	135842.41	
7488	7852	15651581	Lavrentyev	758	Germany	Male	68	6	112595.85	
6865	3941	15659736	Herbert	716	Germany	Male	66	5	121411.90	
6832	1428	15799966	Chigolum	792	Germany	Female	59	9	101609.77	
6890	6319	15686835	Crawford	738	Germany	Female	57	9	148384.64	
7440	6198	15645200	Chiang	581	Germany	Female	54	2	152508.99	
7298	777	15712551	Shen	622	Germany	Female	58	7	116922.25	
6823	599	15637476	Alexandrova	683	Germany	Female	57	5	162448.69	
7456	6813	15605059	Mackie	576	Germany	Male	63	3	148843.56	
7409	5005	15625092	Colombo	502	Germany	Female	57	3	101465.31	
6899	7259	15747757	Trevascus	600	Germany	Female	58	8	118723.11	
7725	3367	15684010	Tuan	640	Germany	Female	74	2	116800.25	
4844	7637	15673238	McCarthy	517	Germany	Female	59	8	154110.99	

In [44]:

```
import pyodbc
import urllib
import sqlalchemy

'''connect to datahub'''

params_datahub = urllib.parse.quote_plus("DRIVER={SQL Server Native Client 11.0};"
                                         "SERVER=localhost\SQLEXPRESS;"
                                         "DATABASE=datahub;"
                                         "UID=sa;"
                                         "PWD=user1")

engine_datahub = sqlalchemy.create_engine("mssql+pyodbc:///?odbc_connect={}".format(params_datahub
# df=df.drop('Purpose_other',axis=1)
df.to_sql('Marketing_prediction',engine_datahub,if_exists='replace',index=False)
```

In [ ]: